Forever Hotel

Software Requirements Specification (SRS)

By

Rashmika Nethsarani
SE/2021/017
Rasadhi Sanchala
SE/2021/021
Hiruni Ramanayaka
SE/2021/024
Viduni Niketha
SE/2021/044
Pubudu Madushan
SE/2021/060

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B. Sc.SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
Sri Lanka
2026

Version History Table

Version No.	Date	Author(s)	Summary of Changes
1.0	2026.03.22	Rashmika Nethsarani Rasadhi Sanchala Hiruni Ramanayaka Viduni Niketha Pubudu Madushan	Initial SRS draft

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Purpose 1
1.2 Scope 2
1.3 Definitions, Acronyms and Abbreviations 3
1.4 Document Overview 4
CHAPTER 2: PROJECT SCOPE 6
2.1 Business Context and Problem Statement 6
2.2 Project Objectives 6
2.3 System Context 7
CHAPTER 3: STAKEHOLDERS 8
CHAPTER 4: FUNCTIONAL REQUIREMENTS 9
4.1 HW – Hotel Website 9
4.2 MAD – Manager Dashboard 10
4.3 FDS – Front Desk System 12
4.4 FOSS – Guest App 14
4.5 KMS – Kitchen Management System 15
4.6 WKMS – Worker Management System 17
CHAPTER 5: NON-FUNCTIONAL REQUIREMENTS 20
CHAPTER 6: BUSINESS RULES 23
6.1 Booking and Reservation Rules 23
6.2 Food Ordering Rules 23
6.3 Task Assignment and Escalation Rules 23
6.4 Access Control Rules 24
CHAPTER 7: CONSTRAINTS 25
7.1 Technical Constraints 25
7.2 Regulatory and Legal Constraints 25
7.3 Operational Constraints 25
CHAPTER 8: ASSUMPTIONS 26
CHAPTER 9: USE CASE DIAGRAMS 27
9.1 Subsystem 01 - Hotel Website 27
9.3 Subsystem 03 - Front Desk System 29
9.4 Subsystem 04 - Food Ordering and Service Request App (FOSS) 30
9.5 Subsystem 05 - Kitchen Management System 31
9.6 Subsystem 06 - Worker Management System (WKMS) 32
CHAPTER 10: USE CASE DESCRIPTIONS 33

10.1 Subsystem 01 - Hotel Website 33
10.1.1 UC-HW-01 - View Hotel Information and Contact Hotel 33
10.1.2 UC-HW-02 - Book Room and Complete Reservation 33
10.1.3 UC-HW-03 - Guest Account and Booking Self-Service 34
10.2 Subsystem 02 - Manager Analytical Dashboard 34
10.2.1 UC-MD-01 - View Analytics and Manage Promotions 34
10.2.2 UC-MD-02 - Manage Staff Accounts 35
10.3 Subsystem 03 - Front Desk System 36
10.3.1 UC-FD-01 - Perform Guest Check-in 36
10.3.2 UC-FD-02 - Guest Checkout, Manage Room Status, and Handle Escalated 37
10.4 Subsystem 04 - Food Ordering and Service Request App (FOSS) 38
10.4.1 UC-FOSS-01 - Access FOSS and Place Food Order 38
10.4.2 UC-FOSS-02 - Submit Service Request and Complaint 39
10.5 Subsystem 05 - Kitchen Management System 39
10.5.1 UC-KMS-01 — Manage Menu Items, Stock Status, Ordering Time Limits 39
10.5.2 UC-KMS-02 - Receive, Process, and Update Food Orders 40
10.6 Subsystem 06 - Worker Management System (WKMS) 41
10.6.1 UC-WKMS-01 — View Shared Task Queue and Self-Assign Task 41
10.6.2 UC-WKMS-02 - Update Task Status and View Shift Performance 41
CHAPTER 11: ACTIVITY DIAGRAMS 43
11.1 Online Room Booking 43
11.2 Guest Check-in 44
11.3 Guest Check-out 45
11.4 Worker Performance Complaint Management 46
11.5 Register Worker 47
11.6 Manager Analytics 48
11.7 Manage Promotions 49
11.8 Service Request 50
11.9 Food Order 51
11.10 Worker Task Claim 52
11.11 Kitchen Order processing 53
CHAPTER 12: EXTERNAL INTERFACE REQUIREMENTS 54
12.1 User Interface Requirements 54
12.1.1 General user interface standards 54
12.1.2 Hotel website (HW) - public-facing interface 54
12.1.3 Manager dashboard (MAD) 55
12.1.4 Front desk system (FDS) 55

12.1.5 Guest app / FOSS — in-room portal 56
12.1.6 Kitchen management system (KMS) 56
12.1.7 Worker management system (WKMS) 56
12.2 Hardware Interface Requirements 57
12.3 Software Interface Requirements 58
12.3.1 External third-party service integrations 58
12.3.2 Internal platform software interfaces 58
12.4 Communications Interface Requirements 60
12.4.1 Network protocol requirements 60
12.4.2 Data formats and standards 61
12.4.3 Authentication and security standards 61
CHAPTER 13: DATA REQUIREMENTS 62
13.1 Data Entities and Attributes 62
13.1.1 Booking 62
13.1.2 Guest 63
13.1.3 Food order 63
13.1.4 Task (service request / housekeeping / delivery) 64
13.2 Data Retention Requirements 68
13.3 Data Integrity and Privacy Requirements 69
CHAPTER 14: ALTERNATIVE SOLUTIONS CONSIDERED 70
14.2 Real-Time Communication: Polling vs WebSockets vs Server-Sent Events 71
14.3 Task Assignment Model: Automatic Assignment vs Self-Assignment Queue 71
14.5 Payment Gateway: Stripe vs Local Payment Processors 73
CHAPTER 15: FEASIBILITY STUDY 74
15.1 Technical Feasibility 74
15.1.1 Technology stack assessment 74
15.1.2 Technical feasibility conclusion 75
15.2 Economic Feasibility 75
15.2.1 Development cost estimate 75
15.2.2 Expected benefits 76
15.3 Operational Feasibility 76
15.3.1 Stakeholder readiness assessment 76
15.3.2 Operational risk factors and mitigations 77
15.4.1 Development timeline overview 78
15.4.2 Schedule risk assessment 78
15.4.3 Schedule feasibility conclusion 79
CHAPTER 16: RISK ANALYSIS 80

16.1 Risk Register 80
16.2 Risk Categories 81
16.2.1 Technical risks 81
16.2.2 Schedule risks 81
16.2.3 Resource risks 82
16.2.4 Requirements risks 82
16.2.5 External and organizational risks 83
CHAPTER 17: REFERENCES 84
Appendices 88
Appendix A: Diagrams 88

LIST OF FIGURES

Figure 1: Use Case Diagram of Hotel Website 27
Figure 2: Use Case Diagram of Manager Analytical Dashboard 28
Figure 3: Use Case Diagram of Front Desk System 29
Figure 4: Use Case Diagram of Food Ordering and Service Request 30
Figure 5: Use Case Diagram of Kitchen Management System 31
Figure 6: Use Case Diagram of Worker Management System 32
Figure 7: Activity Diagram of Online Room Booking 43
Figure 8: Activity Diagram of Guest Check-in 44
Figure 9: Activity Diagram of Guest Check-out 45
Figure 10: Activity Diagram of Worker Performance Complaint Management 46
Figure 11: Activity Diagram of Register Worker 47
Figure 12: Activity Diagram of Manager Analytics 48
Figure 13: Activity Diagram of Manage Promotions 49
Figure 14: Activity Diagram of Service Request 50
Figure 15: Activity Diagram of Food Order 51
Figure 16: Activity Diagram of Worker Task Claim 52
Figure 17: Activity Diagram of Kitchen Order Processing 53

List of Tables

Table 1: Definitions, Acronyms and Abbreviations 4
Table 2: Stakeholder Identification and Classification 8
Table 3: Functional Requirements – Hotel Website 10
Table 4: Functional Requirements – Manager Dashboard 12
Table 5: Functional Requirements – Front Desk System 14
Table 6: Functional Requirements – Guest App (FOSS) 15
Table 7: Functional Requirements – Kitchen Management System 17
Table 8: Functional Requirements – Worker Management System 19
Table 9: Non-Functional Requirements 22
Table 10: Use Case Description HW-01 33
Table 11: Use Case Description HW-02 34
Table 12: Use Case Description HW-03 34
Table 13: Use Case Description MD-01 35
Table 14: Use Case Description MD-02 36
Table 15: Use Case Description FD-01 36
Table 16: Use Case Description FD-02 38
Table 17: Use Case Description FOSS-01 39
Table 18: Use Case Description FOSS-02 39
Table 19: Use Case Description KMS-01 40
Table 20: Use Case Description KMS-02 41
Table 21: Use Case Description WKMS-01 41
Table 22: Use Case Description WKMS-02 42
Table 23: Hardware Interface Requirements 57
Table 24: External Third-Party Service Integrations 58
Table 25: Internal Platform Software Interfaces 59
Table 26: Network Protocol Requirements 60
Table 27: Booking Entity Attributes 63
Table 28: Guest Entity Attributes 63
Table 29: Food Order Entity Attributes 64
Table 30: Task Entity Attributes 65
Table 31: Room Entity Attributes 65
Table 32: Menu Item Entity Attributes 66
Table 33: Service Request Entity Attributes 67
Table 34: Worker Entity Attributes 67
Table 35: Complaint Entity Attributes 68
Table 36: Data Retention Requirements 69
Table 37: Architecture: Monolithic vs Microservices 70
Table 38: Real-Time Communication Approaches Compared 71
Table 39: Guest Access: Native App vs Progressive Web App 72
Table 40: Technology Stack Assessment 75
Table 41: Stakeholder Readiness Assessment 77
Table 42: Development Timeline Overview 78
Table 43: Feasibility Study Summary 79
Table 44: Risk Register 81

CHAPTER 1: INTRODUCTION

Forever Hotel is a production-grade, microservices-based Integrated Hotel Management System[1] developed for Forever City Hotel, located at No.243/3, Makola Road, Kiribathgoda, Sri Lanka. The system is designed to digitise, automate, and unify all core hotel operations — from online room reservations and front desk management to kitchen order processing, in-room guest services, worker task coordination, and managerial analytics.

The hospitality industry increasingly demands seamless digital experiences for both guests and operational staff. Forever City Hotel, recognising the limitations of its existing manual and fragmented processes, commissioned the development of Forever Hotel as a comprehensive solution that would integrate all departments under a single, coherent software ecosystem. The system comprises six independently deployable subsystems, each addressing a distinct operational domain, while communicating through well-defined APIs and an asynchronous event bus [2].

This Software Requirements Specification (SRS) [3] document formally defines the complete set of requirements for Forever Hotel. It is based on the approved Functional Requirements Specification and serves as the contractual baseline for all subsequent design, development, and testing activities. All requirements contained herein have been reviewed and approved by the client representative of Forever City Hotel.

1.1 Purpose

This Software Requirements Specification document serves the following purposes

- To provide a complete, unambiguous, and formally structured description of the functional and non-functional requirements for the Forever Hotel Integrated Hotel Management System.
- To serve as the authoritative agreement between the client (Forever City Hotel), the development team, and the academic supervisor regarding the scope and behaviour of the system.
- To establish the baseline requirements from which the System Design Specification (SDS), implementation, and testing artefacts are derived.

- To document all stakeholder-agreed requirements with unique identifiers, priority levels, and approval status to enable traceability throughout the software engineering lifecycle.

The intended audience for this document includes

- The project development team - for design and implementation guidance.
- The client representative (Forever City Hotel) - for requirements validation and sign-off.
- The academic supervisor - for project assessment and academic oversight.
- Quality assurance engineers -for the derivation of test cases and acceptance criteria.
- Future development and maintenance teams - for system understanding and change management.

1.2 Scope

The system described in this document is named Forever Hotel. It is a microservices-based Integrated Hotel Management System composed of six subsystems

- Hotel Website (HW) - a publicly accessible web application enabling prospective guests to discover the hotel, browse and compare room types, create guest accounts, and complete online reservations with Stripe payment processing.
- Manager Dashboard (MAD) - an internal web application providing the hotel manager with real-time analytics, occupancy and revenue reporting, promotions management, staff account administration, and oversight of all operational departments.
- Front Desk System (FDS) - a reception-facing web application supporting check-in and check-out processing, walk-in bookings, room status management, folio management, and escalated oversight of unclaimed worker tasks.
- Food Ordering & Service Request Application / Guest App (FOSS) - a mobile-first Progressive Web App accessible to checked-in guests for in-room food ordering, housekeeping and maintenance service requests, and complaint submission.
- Kitchen Management System (KMS) - a tablet-optimised application providing kitchen staff with a real-time FIFO order queue, automatic KOT printing, allergy acknowledgement workflows, and menu and stock management capabilities.

- Worker Management System (WKMS) - a mobile-first Progressive Web App providing a self-assignment task queue for housekeeping and maintenance workers, with escalation detection and front-desk override capability.

Forever Hotel shall perform the following functions within its defined scope

- Online room availability checking and booking with payment processing.
- Guest account management and booking self-service (modification and cancellation).
- Front desk operations including check-in, check-out, and walk-in booking.
- Real-time room status tracking and maintenance management.
- In-room food ordering with a live kitchen queue and automated KOT printing.
- Service request submission, real-time tracking, and worker task self-assignment.
- Guest complaint submission and management resolution tracking.
- Manager-level analytics, reporting, and staff account administration.
- System-wide role-based access control with JWT authentication.

The following items are explicitly excluded from the scope of Forever Hotel

- Accounting, payroll, or general ledger integration with external financial systems.
- Inventory management for non-food hotel assets (e.g., linen, furniture, equipment).
- Channel management integration with third-party booking platforms other than Booking.lk.
- Native mobile application development (iOS/Android); all mobile access is delivered via Progressive Web App technology.
- On-site hardware procurement, printer installation, or network infrastructure setup.
- Loyalty programme or points-based reward systems.

1.3 Definitions, Acronyms and Abbreviations

The following table defines all domain-specific terms, acronyms, and abbreviations used throughout this document.

Term / Acronym	Definition
ESC/POS	Escape/Point-of-Sale — thermal printer command language
FDS	Front Desk System
FOSS	Food Ordering and Service Request Application (Guest App)
FR	Functional Requirement

FRS	Functional Requirements Specification
HW	Hotel Website
ISO/IEC 25010	International standard for software product quality
KMS	Kitchen Management System
KOT	Kitchen Order Ticket — printed slip for kitchen preparation
MAD	Manager Analytical Dashboard
NFR	Non-Functional Requirement
NIC	National Identity Card
PCI-DSS	Payment Card Industry Data Security Standard
RBAC	Role-Based Access Control
REST	Representational State Transfer — API architectural style
SDS	System Design Specification
SendGrid	Cloud-based transactional email platform
SPA	Single-Page Application
SRS	Software Requirements Specification
SSL/TLS	Secure Sockets Layer / Transport Layer Security
VPS	Virtual Private Server
WebSocket	Full-duplex communication protocol over a single TCP connection
WKMS	Worker Management System

Table 1: Definitions, Acronyms and Abbreviations

1.4 Document Overview

This Software Requirements Specification is organised into the following chapters:

- Chapter 1 - Introduction: Provides the purpose, scope, definitions, and structural overview of this document. Establishes the system context and client background.
- Chapter 2 - Project Scope: Defines the system boundaries, project objectives, in-scope and out-of-scope functionality, and system context in relation to the hotel's operational environment.
- Chapter 3 - Stakeholders: Identifies all primary, secondary, and tertiary stakeholders, their roles, and their interests in the Forever Hotel system.
- Chapter 4 - Functional Requirements: Presents the complete set of approved functional requirements for all six subsystems, documented with unique identifiers, priority levels, source stakeholders, and approval status.
- Chapter 5 - Non-Functional Requirements: Specifies measurable quality attributes categorised by performance, reliability, usability, security, scalability, maintainability, and portability, aligned with ISO/IEC 25010.

- Chapter 6 - Business Rules: Documents the business logic and operational policies that constrain system behaviour.
- Chapter 7 - Constraints: Describes technical, regulatory, and organisational constraints that bound the design and implementation.
- Chapter 8 - Assumptions: Lists the assumptions made during requirements elicitation that underpin this specification.
- Chapter 9 - Use Case Diagrams: Provides UML use case diagrams for each subsystem, illustrating actor interactions and system boundaries.
- Chapter 10 - Use Case Descriptions: Provides detailed narrative descriptions for all major use cases, including pre-conditions, post-conditions, main flow, and alternative flows.
- Chapter 11 - Activity Diagrams: Presents UML activity diagrams for key operational workflows.
- Chapter 12 - External Interface Requirements: Specifies user interface, hardware interface, software interface, and communications interface requirements.
- Chapter 13 - Data Requirements: Defines the data entities, relationships, and data management requirements.
- Chapter 14 - Alternative Solutions Considered: Documents alternative architectural and technological approaches evaluated during requirements analysis.
- Chapter 15 - Feasibility Study: Assesses technical, economic, operational, and schedule feasibility.
- Chapter 16 - Risk Analysis: Presents a structured Risk Register with likelihood, impact, and mitigation strategies.
- Chapter 17 - References: Lists all external documents, standards, and sources cited.

CHAPTER 2: PROJECT SCOPE

This chapter provides a comprehensive definition of the scope of the Forever Hotel project, establishing the boundaries of the system and the context within which it operates. It describes the business problem that motivated the development of Forever Hotel, the objectives the system is designed to achieve, and the operational domains it encompasses. The chapter concludes by articulating what is explicitly outside the system's scope to prevent scope creep and to establish clear client expectations.

2.1 Business Context and Problem Statement

Forever City Hotel is a mid-scale hospitality establishment in Kiribathgoda, Sri Lanka, offering accommodation, food and beverage services, and a range of recreational and business facilities. Prior to the development of Forever Hotel, the hotel operated with a combination of manual processes and fragmented software tools that were not integrated with one another. This resulted in a number of operational inefficiencies, including delayed check-in and check-out processes, communication gaps between front desk, kitchen, and housekeeping departments, a lack of real-time visibility for hotel management, and an absence of a structured digital channel for guests to interact with hotel services during their stay.

The absence of an online booking facility meant that the hotel was not accessible to prospective guests who prefer to research and book accommodation digitally. Furthermore, without a centralised management dashboard, the hotel manager was unable to monitor operations in real time, identify demand patterns, or take data-driven decisions regarding pricing, staffing, and promotions.Forever Hotel is developed to address these challenges by providing a unified, integrated, and scalable hotel management platform that serves all major operational stakeholders simultaneously.

2.2 Project Objectives

The following objectives define the success criteria for the Forever Hotel project

- To provide an online presence for Forever City Hotel through a fully functional, mobile-responsive hotel website with real-time room availability, online booking, and Stripe payment processing.
- To automate and digitise the front desk workflow, enabling efficient check-in, check-out, walk-in booking, and room management through the Front Desk System.

- To provide hotel guests with a self-service in-room portal (Guest App) for food ordering, service requests, and complaint submission, enhancing the overall guest experience.
- To digitise kitchen operations through a real-time order management system with automated KOT printing, allergy acknowledgement, and menu management.
- To implement a self-assignment task management system for housekeeping and maintenance workers, improving response times and reducing management overhead.
- To provide the hotel manager with a comprehensive analytics dashboard that aggregates data from all operational subsystems, enabling informed decision-making.
- To implement a secure, role-based access control system that ensures each user role accesses only the functionality and data relevant to their responsibilities.
- To design and deploy the system as a containerised microservices architecture, ensuring scalability, fault isolation, and independent deployability of each subsystem.

2.3 System Context

Forever Hotel operates within the hotel's internal network for staff-facing subsystems (MAD, FDS, KMS, WKMS) and over the public internet for guest-facing subsystems (HW, FOSS). The system integrates with the following external platforms

- Stripe — for secure online payment processing[4] for room bookings made via the Hotel Website.
- SendGrid — for transactional email delivery [5], including booking confirmations, cancellation receipts, and staff credential notifications.
- Booking.lk — for importing reservation data from the third-party booking platform into the Front Desk System.
- Network Thermal Printers — for automated Kitchen Order Ticket (KOT) printing via the Printing Service microservice.

All subsystems communicate through a centralised API Gateway that handles request routing, authentication enforcement, and rate limiting. Asynchronous inter-service communication is handled through a RabbitMQ [6] message broker, ensuring that real-time events (such as new food orders, service request escalations, and task completions) are reliably propagated across the system.

CHAPTER 3: STAKEHOLDERS

ID	Stakeholder	Organization	Type	Role / Interest
STK-01	Hotel Manager	Forever City Hotel	Primary	Oversees hotel operations; approves promotions, analytics, staff accounts, and complaint escalations. Primary client representative.
STK-02	Receptionist	Forever City Hotel	Primary	Performs check-in, check-out, walk-in bookings, room management, and task oversight at the front desk.
STK-03	Hotel Guest	External	Primary	Books rooms online, uses the guest portal during stay for food ordering, service requests, and complaints.
STK-04	Kitchen Manager	Forever City Hotel	Primary	Manages food menus, categories, ordering windows, and item stock status within KMS.
STK-05	Kitchen Staff	Forever City Hotel	Primary	Monitors order queue, acknowledges allergen information, updates order status, and coordinates with KOT prints.
STK-06	Worker (Housekeeping / Maintenance)	Forever City Hotel	Primary	Claims and completes tasks from the shared WKMS queue, including room cleaning, service delivery, and maintenance.
STK-07	System Administrator	Forever City Hotel / IT Team	Secondary	Manages platform configuration, user provisioning at system level, server maintenance, and deployment.
STK-09	Project Development Team	University of Kelaniya	Secondary	Designs, develops, and documents the Forever Hotel system across all six subsystems.
STK-10	Stripe	External (Third Party)	Tertiary	Payment gateway provider; processes online room booking payments. No direct interaction with the hotel's internal data.
STK-11	SendGrid	External (Third Party)	Tertiary	Transactional email service; dispatches booking confirmations and staff credential emails.
STK-12	Booking.com	External (Third Party)	Tertiary	Third-party reservation platform; FDS imports guest bookings placed via Booking.lk.

Table 2: Stakeholder Identification and Classification

CHAPTER 4: FUNCTIONAL REQUIREMENTS

4.1 HW – Hotel Website

ID	Description	Priority	Source	Status
HW-01	The system shall display a home page with a hero banner, hotel description, highlights, and active promotional announcements fetched from the Manager Dashboard.	High	STK-01, STK-03	Approved
HW-02	The system shall provide an About Us page with hotel history, facilities, policies, and an embedded Google Maps location.	High	STK-01, STK-03	Approved
HW-15	The system shall display a Services & Facilities page listing all amenities, including a swimming pool, karaoke room, WOK family restaurant, meeting rooms, mini function hall, spa, gym, honeymoon packages, and scenic photo locations.	High	STK-01, STK-03	Approved
HW-16	The system shall include an FAQ section and a contact form that emails enquiries to hotel management.	Medium	STK-01, STK-03	Approved
HW-17	The system shall be fully responsive across mobile (320 px+), tablet, and desktop viewports.	High	STK-03	Approved
HW-03	The system shall display a Rooms page listing all room types with high-resolution images, full amenity lists, price per night, and real-time availability indicators.	High	STK-03	Approved
HW-04	The system shall allow guests to compare up to three room types side by side.	Medium	STK-03	Approved
HW-05	The system shall provide an availability calendar for each room type, visually indicating available and blocked dates.	High	STK-03	Approved
HW-09	The system shall validate real-time availability before confirming a booking and reject requests for unavailable room types.	High	STK-03	Approved
HW-06	The system shall allow visitors to self-register a guest account using name, email, and password, stored as a bcrypt hash.	High	STK-03	Approved
HW-07	The system shall allow registered guests to log in securely using email and password, receiving a JWT session token [7].	High	STK-03	Approved
HW-13	The system shall allow logged-in guests to view their upcoming and past bookings in a personal dashboard.	High	STK-03	Approved
HW-08	The system shall allow logged-in guests to book a room by selecting check-in/check-	High	STK-03	Approved

	out dates, room type, number of guests, and optional special requests.			
HW-10	The system shall allow guests to apply a promotional code at checkout; valid codes created in the Manager Dashboard shall reduce the booking total by the configured percentage or fixed amount.	High	STK-01, STK-03	Approved
HW-11	The system shall integrate with Stripe to securely process full payments or advance deposit payments at booking.	High	STK-01, STK-03	Approved
HW-12	The system shall send a booking confirmation email via SendGrid to the guest upon successful payment, including Booking Reference ID, room type, dates, and total paid.	High	STK-03	Approved
HW-14	The system shall allow guests to modify or cancel a booking up to 24 hours before check-in, subject to the hotel cancellation policy, and send a confirmation email for the change.	High	STK-03	Approved

Table 3: Functional Requirements – Hotel Website

4.2 MAD – Manager Dashboard

ID	Description	Priority	Source	Status
MD-01	The system shall display total bookings for the current day, week, and month with trend line and bar charts.	High	STK-01	Approved
MD-02	The system shall visualise occupancy rates using a colour-coded calendar heatmap highlighting low-demand and high-demand dates.	High	STK-01	Approved
MD-03	The system shall show revenue summaries (daily, weekly, monthly) with percentage comparison to the previous equivalent period.	High	STK-01	Approved
MD-04	The system shall identify low-booking days and display an alert suggesting the manager create a promotion.	High	STK-01	Approved
MD-05	The system shall allow the manager to create a promotional code by specifying code string, discount type, discount value, validity period, applicable room types, and maximum redemption count.	High	STK-01	Approved
MD-05b	The system shall allow the manager to activate, deactivate, edit, and delete existing promotional codes.	High	STK-01	Approved

MD-05c	Active promotional codes shall be published in real time to the Hotel Website for guest application at checkout.	High	STK-01, STK-03	Approved
MD-06	The system shall display food order analytics including total orders per day/week/month, most-ordered items, and peak ordering hours.	High	STK-01	Approved
MD-07	The system shall show a real-time kitchen order status overview across pending, in-preparation, ready-for-delivery, and delivered states.	High	STK-01	Approved
MD-08	The system shall display service request analytics including total requests per period, average resolution time, and count of currently unresolved requests.	High	STK-01	Approved
MD-09	The system shall maintain a complaint register showing complaint ID, category, guest description, referenced worker (if any), submission date, and resolution status.	High	STK-01	Approved
MD-09b	The system shall allow the manager to update complaint status and add internal resolution notes.	High	STK-01	Approved
MD-10	The system shall present a worker performance summary including tasks completed per worker per period, average task completion time, and count of open complaints referencing that worker.	High	STK-01	Approved
MD-11	The system shall allow the manager to export any report (bookings, revenue, staff performance) as PDF or CSV.	Medium	STK-01	Approved
MD-12	The system shall display a real-time room status board showing each room's state: Occupied, Vacant, Under Maintenance, or Requires Cleaning.	High	STK-01	Approved
MD-13	The system shall display a notification panel for unresolved complaints and low-stock/unavailable item alerts from the Kitchen Management System.	High	STK-01	Approved
MD-14	The system shall allow the manager to create Receptionist user accounts by entering name, vocation, email, age, phone, and NIC; the system shall auto-generate a username and temporary password, notifying the user via email.	High	STK-01	Approved
MD-15	The system shall allow the manager to create Worker user accounts with the same registration details and credentials delivery process.	High	STK-01	Approved

MD-15b	The system shall allow the manager to create Kitchen Staff user accounts with role Kitchen Staff or Kitchen Manager, with credentials delivered via email.	High	STK-01	Approved
MD-16	The system shall allow the manager to reset the password for any staff account, triggering a new credentials email.	High	STK-01	Approved
MD-16b	The system shall allow the manager to edit staff profiles, change roles, and deactivate accounts.	High	STK-01	Approved
MD-17	The system shall provide a read-only view of the Worker Management System task queue, showing all unassigned, in-progress, and completed tasks in real time.	High	STK-01	Approved

Table 4: Functional Requirements – Manager Dashboard

4.3 FDS – Front Desk System

ID	Description	Priority	Source	Status
FD-01	The system shall allow receptionists to log in with username and password credentials provisioned by the manager via the Manager Dashboard.	High	STK-02	Approved
FD-02	The system shall display a daily arrivals panel listing all expected check-ins and a departures panel listing all expected check-outs for the current date.	High	STK-02	Approved
FD-03	The system shall allow receptionists to search for bookings by guest name, Booking Reference ID, NIC/passport number, email, or phone number.	High	STK-02	Approved
FD-04	The system shall support walk-in bookings: the receptionist creates a new booking by entering guest details, selecting room type and dates, and processing payment (cash or card).	High	STK-02	Approved
FD-05	During check-in, the system shall allow the receptionist to verify guest ID by uploading a scanned copy or marking verification as complete against the physical document presented.	High	STK-02	Approved
FD-06	The system shall assign a specific room number to the booking (if not pre-assigned) and mark the room status as Occupied in the Room Service.	High	STK-02	Approved
FD-07	Upon successful check-in, the system shall automatically activate the guest's session access to the Guest App (FOSS) for the	High	STK-02, STK-03	Approved

	duration of their stay, issuing a room-linked session token.			
FD-08	The system shall allow the receptionist to print a registration card and payment receipt upon check-in.	Medium	STK-02	Approved
FD-09	During check-out, the system shall display a full itemised folio of all outstanding charges (room charges, food orders, additional services, unpaid balance) and accept final payment.	High	STK-02, STK-03	Approved
FD-10	Upon check-out, the system shall mark the room as Requires Cleaning, deactivate the guest's FOSS session, and automatically create a room-cleaning task in the WKMS task queue with High priority.	High	STK-02	Approved
FD-11	The system shall allow room changes: the receptionist may reassign a guest to a different room, updating both rooms' statuses accordingly.	Medium	STK-02	Approved
FD-12	The system shall display a real-time room status board showing each room's current state: Clean/Vacant, Occupied, Requires Cleaning, or Under Maintenance.	High	STK-02	Approved
FD-13	The system shall allow the receptionist to mark a room as Under Maintenance, blocking it from future bookings until cleared.	High	STK-02	Approved
FD-14	The system shall allow the receptionist to create a service request on behalf of a guest (e.g., extra towels, maintenance), submitting it directly to the WKMS task queue.	Medium	STK-02, STK-03	Approved
FD-15	The system shall allow the receptionist to view a guest's running folio/charges at any point during the stay.	Medium	STK-02	Approved
FD-16	The system shall log all front desk actions (check-in, check-out, room changes, task assignments) to a tamper-evident audit log.	High	STK-01, STK-02	Approved
FD-17	The system shall display a real-time escalation panel showing all worker tasks that have remained unclaimed for more than 15 minutes, highlighted in red.	High	STK-02	Approved
FD-18	The system shall allow the receptionist to manually assign any unclaimed service request to a specific available worker, updating the task in WKMS immediately.	High	STK-02	Approved
FD-19	The system shall receive escalation alert events from WKMS via the event bus and display a notification badge on the front	High	STK-02	Approved

	desk dashboard whenever a new escalation occurs.			

Table 5: Functional Requirements – Front Desk System

4.4 FOSS – Guest App

ID	Description	Priority	Source	Status
GA-01	The system shall restrict access to guests with an active, checked-in booking session; all other access attempts shall be rejected.	High	STK-03	Approved
GA-01b	The system shall automatically deactivate guest access upon check-out or when the receptionist revokes the session from the Front Desk System.	High	STK-02, STK-03	Approved
GA-02	The system shall display separate menus for Breakfast, Lunch, Dinner, and Tea-Time, showing only items that are currently Available in KMS and within the ordering time window for their meal period.	High	STK-03	Approved
GA-03	Each menu item shall show: name, description, ingredients, high-resolution image, calorie count, allergen tags, dietary labels (vegan, gluten-free, etc.), portion limit, and price.	High	STK-03	Approved
GA-04	The system shall allow guests to add items to a cart, specifying quantity per item subject to per-item portion limits set by the hotel.	High	STK-03	Approved
GA-05	The system shall provide a field to specify allergy information and dietary preferences per order (maximum 300 characters).	High	STK-03	Approved
GA-06	The system shall enforce meal-specific cutoff times and reject or warn the guest if they attempt to order past the cutoff.	High	STK-03, STK-04	Approved
GA-07	The system shall allow guests to cancel a placed order within 10 minutes of submission; after 10 minutes the Cancel button is disabled and a notification is displayed.	High	STK-03	Approved
GA-08	The system shall display an estimated delivery time after order confirmation, calculated by KMS based on the current preparation queue.	High	STK-03	Approved
GA-09	The system shall maintain an order history for the current stay, showing past orders with their final status (Delivered or Cancelled).	Medium	STK-03	Approved

GA-10	The system shall allow guests to submit a service request by selecting a category (Room Cleaning, Extra Towels, Water Bottles, Maintenance, Laundry Pickup, Other) and adding an optional description.	High	STK-03	Approved
GA-11	The system shall automatically associate the service request with the guest's room number without requiring manual entry.	High	STK-03	Approved
GA-12	The system shall display a real-time request status tracker with states: Submitted → Assigned → In Progress → Completed.	High	STK-03	Approved
GA-13	The system shall allow guests to submit a complaint by selecting a category, entering a description, optionally referencing a specific order or service request ID, and selecting a severity level (Low, Medium, High).	High	STK-03	Approved
GA-14	The system shall generate a unique complaint reference number and display it to the guest upon submission.	High	STK-03	Approved
GA-15	The system shall allow guests to view the current status of their submitted complaints.	Medium	STK-03	Approved

Table 6: Functional Requirements – Guest App (FOSS)

4.5 KMS – Kitchen Management System

ID	Description	Priority	Source	Status
KM-01	The system shall display a real-time Kanban-style order queue with three columns: New Orders, In Preparation, and Ready for Delivery.	High	STK-05	Approved
KM-02	Orders shall be sorted strictly by submission timestamp (FIFO) within each queue column.	High	STK-05	Approved
KM-03	Each order card shall show: Order ID, Room Number, Meal period, list of items with quantities, allergy annotations (highlighted in red), submission time, and elapsed waiting time.	High	STK-05	Approved
KM-04	The system shall require kitchen staff to acknowledge allergy and dietary information via a confirmation pop-up modal before moving an order to In Preparation.	High	STK-04, STK-05	Approved
KM-05	Kitchen staff shall update order status using one-click controls: New Order → In Preparation → Ready for Delivery.	High	STK-05	Approved
KM-05b	When a kitchen staff member moves an order to Ready, the system shall notify	High	STK-05, STK-06	Approved

	WKMS via the event bus to dispatch a delivery worker to the specified room.			
KM-05c	The Delivered status shall be set by the delivery worker via WKMS; KMS shall reflect this final status update.	High	STK-05, STK-06	Approved
KM-06	When a new order is placed, the system shall automatically send a Kitchen Order Ticket (KOT) to the designated kitchen printer via the Printing Service (ESC/POS or IP printing).	High	STK-05	Approved
KM-07	If a guest cancels an order within the 10-minute cancellation window, the system shall reprint the KOT with a prominent CANCELLED overlay and update the order status to Cancelled.	High	STK-05	Approved
KM-08	The system shall display a meal-period summary showing total orders pending, in preparation, and completed for the current meal period.	Medium	STK-05	Approved
KM-09	The system shall show the total quantity of each menu item ordered during the current meal period.	Medium	STK-05	Approved
KM-10	The system shall provide an hourly order forecast view based on pre-placed orders to assist kitchen preparation planning.	Low	STK-04, STK-05	Approved
KM-11	The system shall trigger an audio chime and a visual flash notification for each new order received.	High	STK-05	Approved
KM-12	The system shall allow staff to search completed orders by date, room number, or order ID.	Medium	STK-05	Approved
KM-13	The system shall calculate an estimated preparation time for each order based on current queue depth and historical average prep time per item type, and send this estimate to the Guest App.	High	STK-03, STK-05	Approved
KM-M01	The system shall allow the Kitchen Manager to create meal categories (Breakfast, Lunch, Dinner, Tea-Time, Special Meals) and set the ordering time window for each.	High	STK-04	Approved
KM-M02	The system shall allow the Kitchen Manager to add menu items under each category with: name, description, price, preparation time, allergen tags, dietary labels, item image, and per-guest portion limit.	High	STK-04	Approved
KM-M03	The system shall allow the Kitchen Manager to edit any attribute of an existing menu item.	High	STK-04	Approved

KM-M04	The system shall allow kitchen staff to set item stock status to one of four states: Available, Low Stock, Unavailable, or Temporarily Disabled.	High	STK-04, STK-05	Approved
KM-M05	Items marked Unavailable or Temporarily Disabled shall be hidden from the guest menu in FOSS in real time; Low Stock items shall remain visible with a Limited Availability badge.	High	STK-03, STK-04	Approved
KM-M06	Stock status changes shall be propagated to the Manager Dashboard in real time, triggering a low-stock alert in the manager notification panel.	High	STK-01, STK-04	Approved
KM-AC1	The system shall enforce role-based access: Kitchen Manager (full access including menu and category management) and Kitchen Staff (order queue management only, no menu editing).	High	STK-04, STK-05	Approved
KM-AC2	All kitchen staff accounts shall be provisioned by the Manager via the Manager Dashboard.	High	STK-01, STK-04	Approved

Table 7: Functional Requirements – Kitchen Management System

4.6 WKMS – Worker Management System

ID	Description	Priority	Source	Status
WM-01	The system shall display a shared real-time task queue visible to all logged-in workers, containing all unassigned service requests ordered by submission time (oldest first).	High	STK-06	Approved
WM-01b	The queue shall update in real time via WebSocket when new tasks arrive from FOSS or are claimed by another worker.	High	STK-06	Approved
WM-01c	Food delivery tasks emitted by KMS when an order reaches Ready status shall appear in the queue as a Delivery task type with the room number and order ID.	High	STK-06	Approved
WM-02	Each task card shall show: Request ID, Room Number, Category, Guest description, Submission timestamp, and Elapsed waiting time.	High	STK-06	Approved
WM-03	Any logged-in worker may claim an unassigned task by tapping the Claim button on its task card.	High	STK-06	Approved
WM-04	The system shall lock a claimed task to the claiming worker immediately using an optimistic UI update and a database-level status flag, preventing race conditions and	High	STK-06	Approved

	removing the task from all other workers' queues atomically.			
WM-05	A worker may hold a maximum of three simultaneously active tasks (configurable by the manager); the system shall enforce this limit and disable the Claim button when the limit is reached.	High	STK-01, STK-06	Approved
WM-06	Workers shall see a My Tasks panel showing all tasks currently assigned to them, with In Progress and Mark Complete status controls on each card.	High	STK-06	Approved
WM-06b	When a worker marks a task as Completed, the completion event (with timestamp and worker ID) shall be written to the shared database and consumed by the Analytics Service for Manager Dashboard performance reporting.	High	STK-01, STK-06	Approved
WM-07	The system shall escalate any task unclaimed for more than 15 minutes by highlighting its card in red in all workers' queues.	High	STK-02, STK-06	Approved
WM-07b	Upon escalation threshold breach, the system shall emit an alert event to the Front Desk System dashboard via the event bus, triggering a notification badge on the FDS escalation panel.	High	STK-02	Approved
WM-07c	The escalation timer shall be implemented as a server-side cron job that runs every five minutes, queries the Task Service for unclaimed tasks exceeding 15 minutes, and emits escalation events for each.	High	STK-07	Approved
WM-08	The Front Desk System may manually assign any unassigned or escalated task to a specific worker; the assigned task shall be locked to that worker and removed from the shared queue.	High	STK-02, STK-06	Approved
WM-09	Workers shall be able to view their completed task history for the current shift.	Medium	STK-06	Approved
WM-10	The system shall log all task assignments, status transitions, and completions with timestamps for audit and performance tracking.	High	STK-01, STK-07	Approved
WM-11	Room-cleaning tasks triggered by FDS after check-out shall appear in the queue with a High Priority badge and sorted above standard requests.	High	STK-02, STK-06	Approved
WM-P01	The system shall be implemented as a Progressive Web App (PWA) so workers	High	STK-06	Approved

	can access it on any mobile device over hotel Wi-Fi without installing a native app.			
WM-P02	The system shall cache each worker's My Tasks list for offline viewing during brief network interruptions; status updates shall be queued locally and synced when connectivity is restored.	Medium	STK-06	Approved

Table 8: Functional Requirements – Worker Management System

CHAPTER 5: NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements are categorized and measurable, aligned with ISO/IEC 25010 [8].

NFR ID	Category	Requirement	Acceptance Criteria
NFR-01	Performance	The system shall respond quickly to user requests across all subsystems.	95% of API requests shall respond within 500 ms under normal load (100 concurrent users).
NFR-02	Performance	Real-time events between microservices must be processed efficiently.	Order, task, and notification events shall propagate across services within 1 second.
NFR-03	Performance	Real-time UI updates shall be reflected instantly across dashboards.	Dashboard updates (orders, tasks, room status) shall appear within 2 seconds via WebSocket.
NFR-04	Reliability	The system shall maintain stable operation with minimal downtime.	System availability shall be 99.5% uptime per month excluding scheduled maintenance.
NFR-05	Reliability	Order processing must not be lost even during failures.	100% of submitted food orders shall be persisted in the database and recoverable after system failure or restart.
NFR-06	Reliability	Services shall continue functioning even if one subsystem fails.	Failure of a microservice shall not affect other services due to container isolation.
NFR-07	Usability	Guest applications must be simple and mobile friendly.	The Guest App (FOSS) shall be usable on devices with minimum screen width of 320px without layout issues.
NFR-08	Usability	QR-based access must be simple and efficient for guests.	Guests shall access the web application within 5 seconds after scanning the QR code

			under normal network conditions.
NFR-09	Usability	Workers must be able to operate the task queue quickly.	A worker shall be able to claim a task in less than 3 seconds after opening the queue.
NFR-10	Security	All communications between clients and servers shall be secure.	All external connections must use HTTPS with TLS 1.2 or higher [9].
NFR-11	Security	Guest sessions must be securely isolated per room.	Each guest session token shall be uniquely tied to a room and automatically expire upon checkout or after 24 hours of inactivity.
NFR-12	Security	Role-based access control shall protect system features.	All authenticated endpoints shall validate JWT tokens and enforce RBAC policies.
NFR-13	Security	Sensitive data must not be stored insecurely.	Payment card data shall not be stored in the hotel database and shall be processed only via Stripe.
NFR-14	Scalability	The system shall support increased user load during peak hotel operations.	The system shall support at least 300 concurrent users without performance degradation.
NFR-15	Scalability	The system shall handle peak meal-time ordering loads.	The system shall process at least 100 concurrent food orders with no more than 2% failure rate.
NFR-16	Maintainability	System updates must be manageable without service interruption.	Database schema updates shall be implemented through version-controlled migration scripts.
NFR-17	Maintainability	Logs shall support debugging and auditing.	All services shall generate structured logs retained for at least 12 months.
NFR-18	Maintainability	System errors must be traceable for debugging.	100% of system errors shall be logged with timestamp, service

			name, and detailed error description.
NFR-19	Portability	The system shall be deployable across multiple environments.	All services must run within Docker containers on Linux servers.
NFR-20	Portability	Workers must access the system on different mobile devices.	WKMS shall operate as a Progressive Web Application compatible with modern browsers.
NFR-21	Reliability	Kitchen printing must be reliable for all food orders.	100% of food orders shall generate a printed KOT within 3 seconds of order placement.

Table 9: Non-Functional Requirements

CHAPTER 6: BUSINESS RULES

6.1 Booking and Reservation Rules

1. A room booking must have valid check-in and check-out dates where check-out is later than check-in.
2. Guests may modify or cancel bookings up to 24 hours before check-in.
3. A booking is confirmed only after successful payment confirmation from Stripe.
4. A room cannot be assigned to more than one active booking for the same date range.
5. Walk-in guests must provide valid identification before check-in.
6. Upon check-out, the system must generate a final bill including room charges and food/service orders.

6.2 Food Ordering Rules

1. Guests shall access the food ordering system by scanning the QR code provided in the room, which opens the web-based Guest App.
2. Food orders shall trigger an audio chime and a visual flash notification on the Kitchen Management System display for kitchen staff. All orders shall also be printed as a Kitchen Order Ticket (KOT) via the kitchen thermal printer.
3. Once an order is placed, it shall be immediately sent to the kitchen printer.
4. If an order is cancelled within 10 minutes, a CANCELLED KOT must be reprinted.
5. Guests shall receive an estimated delivery time displayed at the time of order confirmation. Guests shall not receive live step-by-step kitchen preparation updates; however, real-time status tracking is provided for service requests submitted via the Guest App.
6. Estimated delivery time shall be displayed at the time of order confirmation.
7. Orders once moved to “In Preparation” by the kitchen cannot be cancelled under any circumstances.

6.3 Task Assignment and Escalation Rules

1. Service requests submitted by guests are added to the shared task queue in WKMS.
2. Workers may claim tasks voluntarily from the queue.
3. A worker may hold a maximum of three active tasks simultaneously. This limit is configurable by the hotel manager via the Manager Dashboard.

4. Tasks not claimed within 15 minutes must be marked as Escalated.
5. Escalated tasks are displayed on the Front Desk escalation panel.
6. Receptionists may manually assign escalated tasks to available workers.

6.4 Access Control Rules

1. System access must follow Role-Based Access Control (RBAC).
2. Only Managers may create or modify staff accounts.
3. Receptionists may manage check-in, check-out, and room assignments.
4. Kitchen staff may only manage food orders and menu items.
5. Workers may only access the task queue and their assigned tasks.
6. Guests may only access FOSS during their active hotel stay.

CHAPTER 7: CONSTRAINTS

7.1 Technical Constraints

- The system must use a microservices architecture with containerized services.
- Backend services must be implemented using NestJS.
- Frontend interfaces must be implemented using Next.js [10].
- The system must use PostgreSQL as the primary database.
- Inter-service communication must use REST APIs and RabbitMQ messaging.
- The system must support WebSocket-based real-time communication.
- The system must support QR code-based access without requiring mobile app installation.
- The system must support event-driven communication using a message broker (RabbitMQ) for inter-service coordination.

7.2 Regulatory and Legal Constraints

- Payment processing must comply with PCI-DSS standards.
- Guest personal data must be stored in accordance with data protection regulations.
- Email communications must comply with international email service regulations.

7.3 Operational Constraints

- The system must operate within the hotel’s internal Wi-Fi network.
- Kitchen printing must support ESC/POS compatible thermal printers.
- The system must support mobile devices used by hotel staff.
- Each room must display a unique QR code linked to the guest session system.
- Kitchen operations depend on continuous printer availability.

CHAPTER 8: ASSUMPTIONS

1. The hotel has stable internet connectivity and internal Wi-Fi coverage.
2. Staff members have basic digital literacy to operate the system.
3. Guests have mobile devices capable of scanning QR codes.
4. The hotel will maintain dedicated hardware such as printers and servers.
5. The external services (Stripe, SendGrid, Booking.lk API) will remain operational.
6. The hotel management will provide training to staff before deployment.
7. Guest devices will have modern web browsers capable of running Progressive Web Applications.

CHAPTER 9: USE CASE DIAGRAMS

9.1 Subsystem 01 - Hotel Website

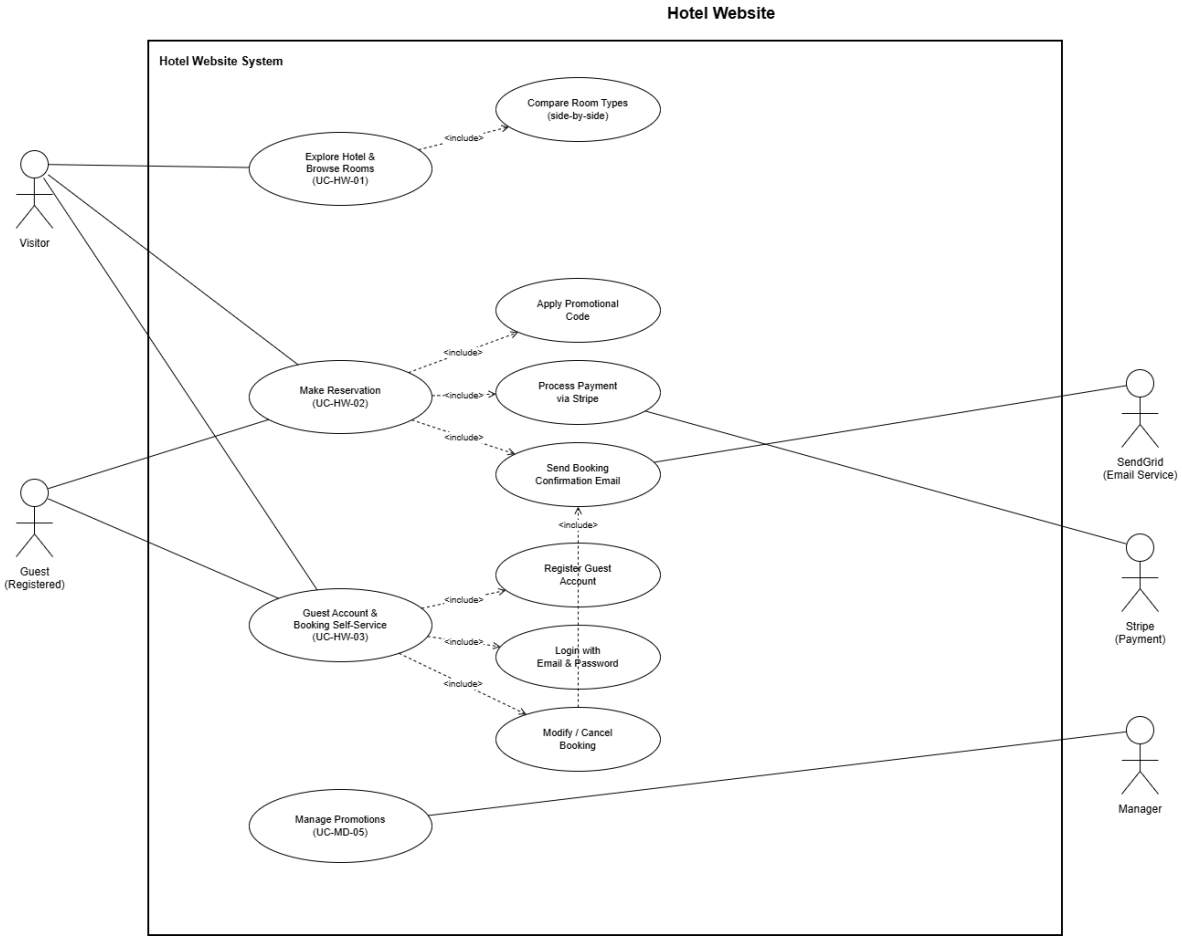


Figure 1: Use Case Diagram of Hotel Website

9.2 Subsystem 02 - Manager Analytical Dashboard

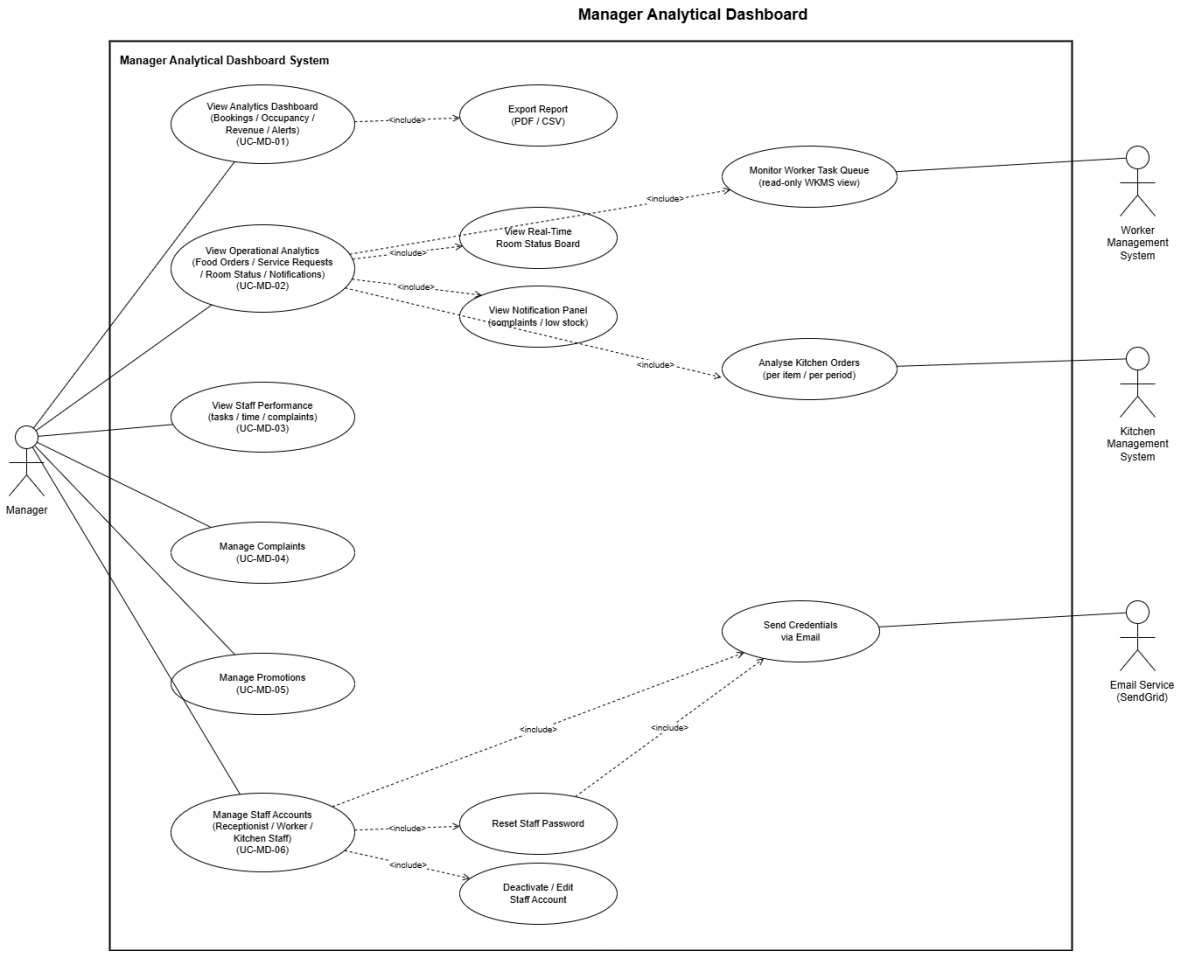


Figure 2: Use Case Diagram of Manager Analytical Dashboard

9.3 Subsystem 03 - Front Desk System

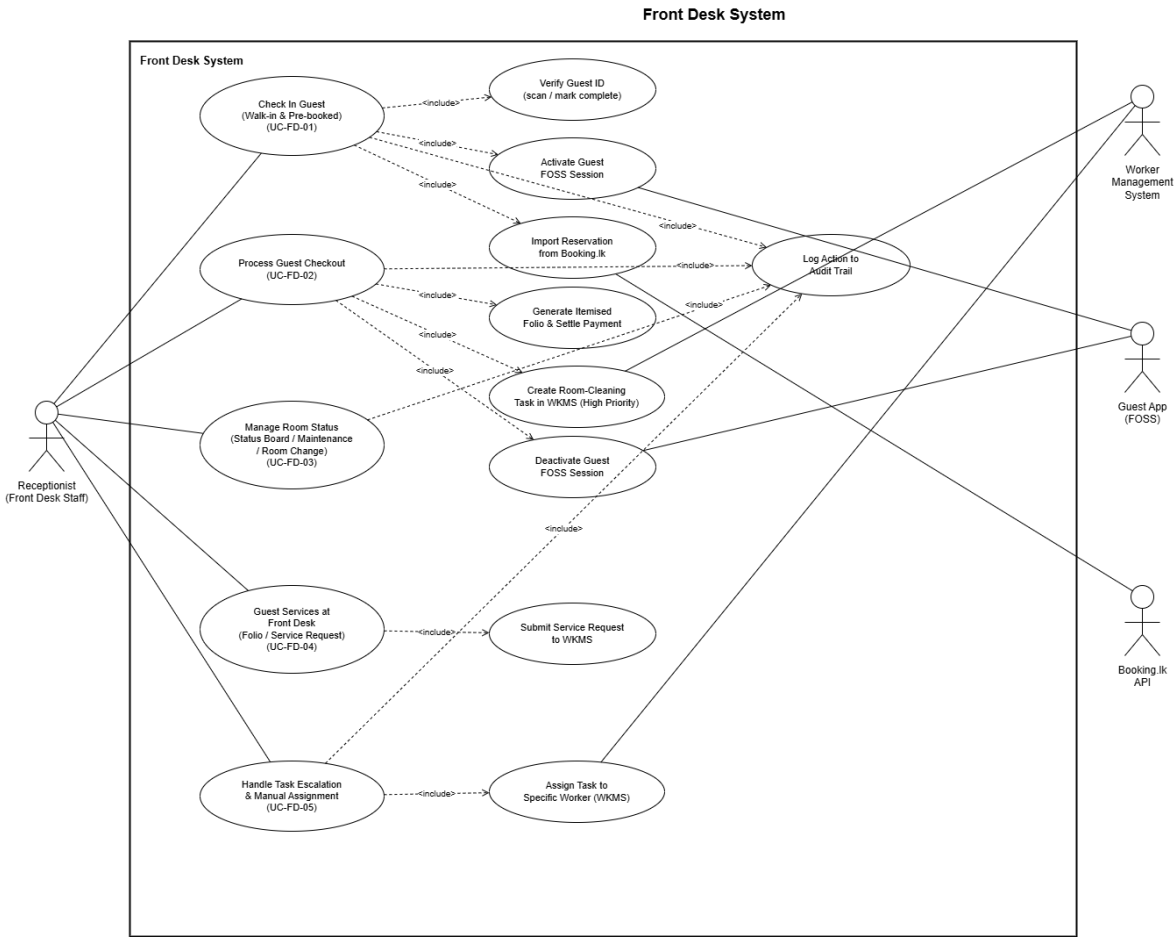


Figure 3: Use Case Diagram of Front Desk System

9.4 Subsystem 04 - Food Ordering and Service Request App (FOSS)

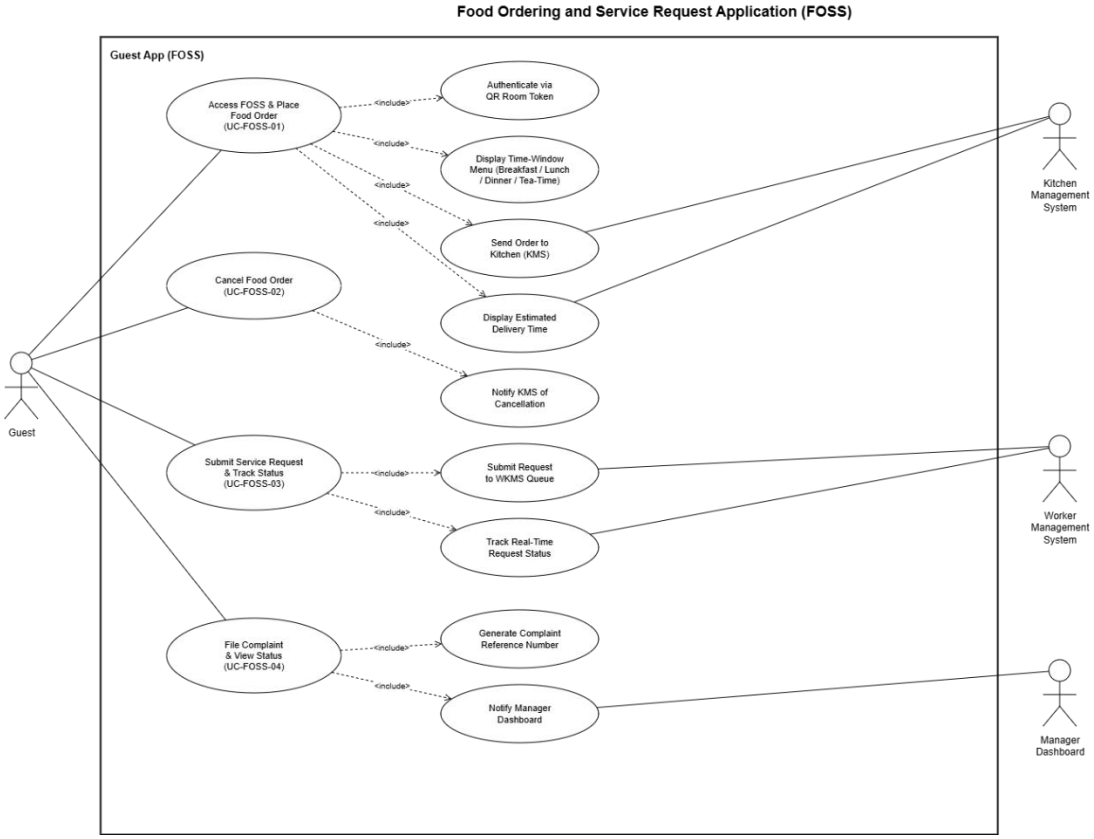


Figure 4: Use Case Diagram of Food Ordering and Service Request

9.5 Subsystem 05 - Kitchen Management System

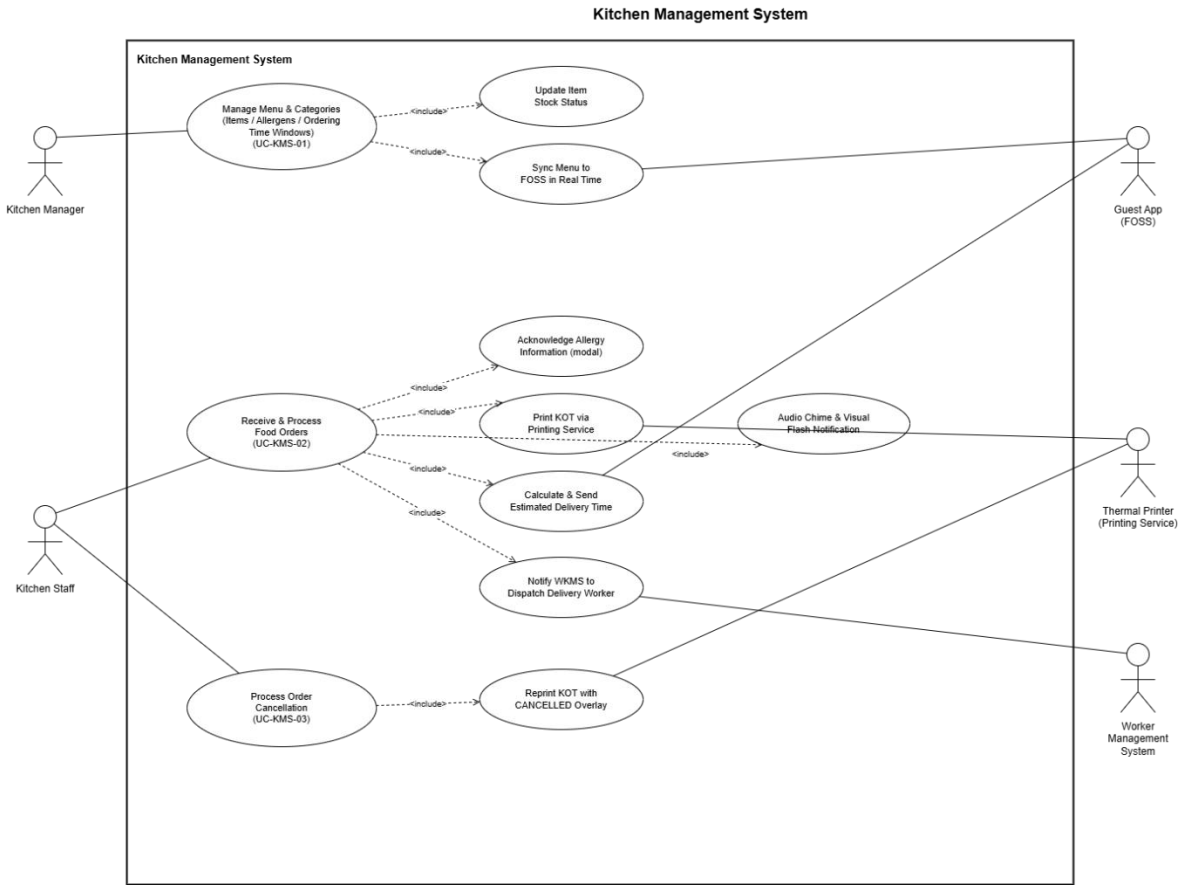


Figure 5: Use Case Diagram of Kitchen Management System

9.6 Subsystem 06 - Worker Management System (WKMS)

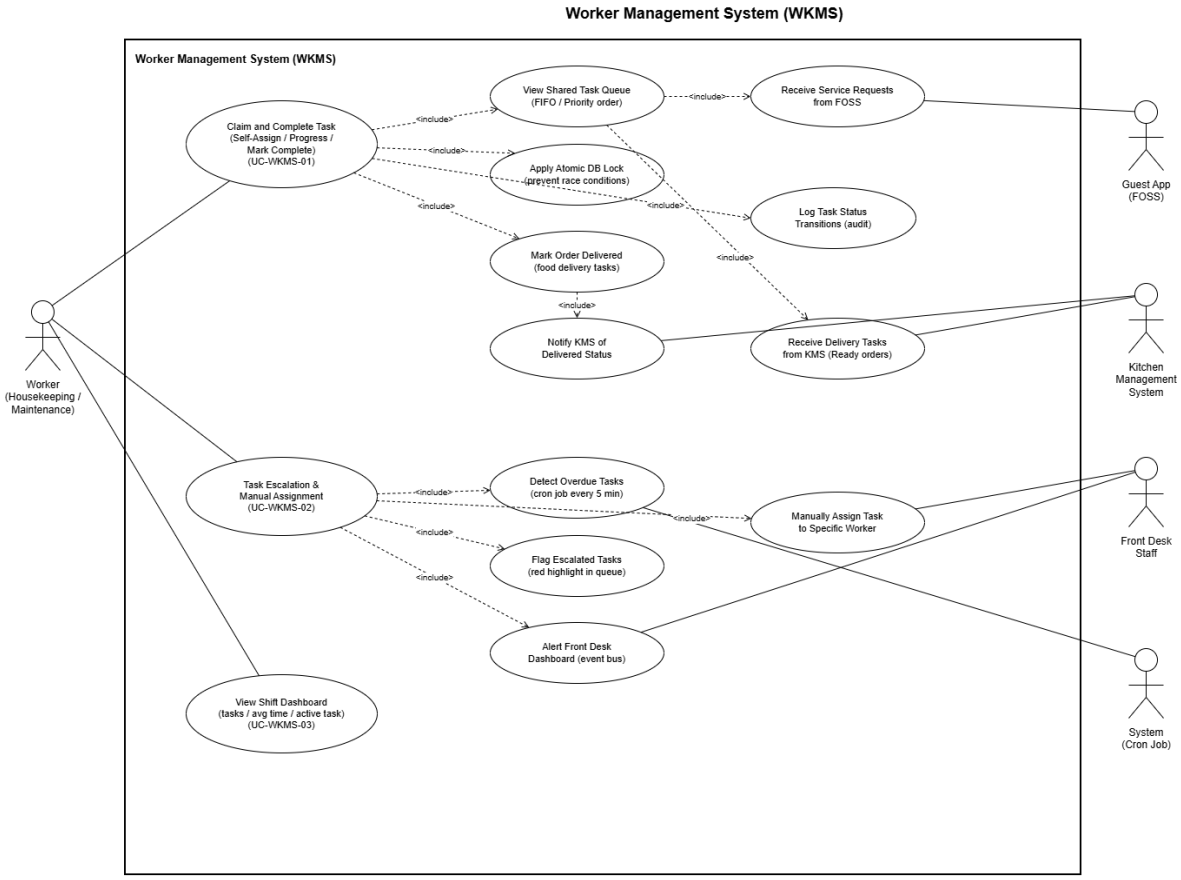


Figure 6: Use Case Diagram of Worker Management System

CHAPTER 10: USE CASE DESCRIPTIONS

10.1 Subsystem 01 - Hotel Website

10.1.1 UC-HW-01 - View Hotel Information and Contact Hotel

Use Case ID	UC-HW-01
Use Case Name	View Hotel Information and Contact Hotel
Actor(s)	Visitor
Description	Hotel information shall be presented to the visitor, including home page highlights, services and facilities, promotional content, FAQs, contact details, and location. An inquiry may be submitted by the visitor through the contact form.
Pre-conditions	• The hotel website shall be accessible via a web browser. • Internet connectivity shall be available to the visitor.
Post-conditions	• Hotel information shall have been viewed by the visitor. • Optional: An inquiry shall have been submitted to hotel management.
Main Flow	1. The hotel website shall be accessed by the visitor. 2. The home page shall be displayed by the system, including navigation menus, promotional highlights (fetched from Manager Dashboard), and key hotel information. 3. A section (Services & Facilities, About Us, Promotions, FAQ, Contact, or Location) shall be selected by the visitor. 4. The selected content shall be loaded and displayed by the system, including relevant details, images, embedded map, or forms. 5. If the Contact section is selected, inquiry details shall be entered and submitted by the visitor through the contact form. 6. The submitted inquiry shall be validated and forwarded to hotel management by the system.

Table 10: Use Case Description HW-01

10.1.2 UC-HW-02 - Book Room and Complete Reservation

Use Case ID	UC-HW-02
Use Case Name	Book Room and Complete Reservation
Actor(s)	Registered Guest
Description	A room reservation shall be completed by the registered guest by browsing available rooms, selecting dates and room type, applying optional promotional codes, and processing payment via Stripe. Real-time availability shall be validated before confirmation, and a booking confirmation email shall be sent via SendGrid.
Pre-conditions	• The hotel website shall have been accessed. • The guest shall be authenticated (logged in or registered).
Post-conditions	• The reservation shall have been recorded in the system. • Room availability shall have been updated. • A booking confirmation email shall have been sent to the guest via SendGrid. • The reservation shall be visible in the guest personal dashboard.

Main Flow	1. The Rooms page shall be accessed by the guest; available room types with images, amenities, pricing, and real-time availability shall be displayed by the system. 2. Check-in and check-out dates shall be entered by the guest; the availability calendar shall be validated by the system. 3. A room type shall be selected by the guest; up to three rooms may be compared side by side. 4. Guest details, number of guests, and optional special requests shall be entered. 5. An optional promotional code shall be applied; the discount shall be validated and the total updated by the system. 6. A payment option (full payment or advance deposit) shall be selected; payment shall be processed by the system via Stripe. 7. Upon successful payment, the reservation shall be confirmed, a Booking Reference ID generated, and a confirmation email dispatched by the system. 8. The guest may modify or cancel the booking from the personal dashboard up to 24 hours before check-in.

Table 11: Use Case Description HW-02

10.1.3 UC-HW-03 - Guest Account and Booking Self-Service

Use Case ID	UC-HW-03
Use Case Name	Book Room and Complete Reservation
Actor(s)	Visitor / Guest
Description	A visitor registers for a guest account or logs in to an existing account. Once authenticated, the guest can view upcoming and past bookings and modify or cancel a booking subject to the cancellation policy.
Pre-conditions	• The hotel website shall be accessible via a web browser. • For login and booking management: a guest account must exist.
Post-conditions	• Guest account is created and accessible (registration path). • Guest is authenticated with a JWT session token (login path). • Booking is modified or cancelled and a confirmation email is sent (self-service path).
Main Flow	1. Logged-in guest opens the My Bookings dashboard. 2. System displays upcoming and past bookings with dates, room type, and status. 3. Guest selects a booking to modify or cancel. 4. System checks that the check-in date is more than 24 hours away. 5. Guest confirms the modification or cancellation. 6. System updates the booking record and sends a confirmation email for the change.

Table 12: Use Case Description HW-03

10.2 Subsystem 02 - Manager Analytical Dashboard

10.2.1 UC-MD-01 - View Analytics and Manage Promotions

Use Case ID	UC-MD-01
Use Case Name	View Analytics and Manage Promotions
Actor(s)	Manager

Description	The manager shall view operational analytics covering bookings, occupancy, revenue, food orders, service requests, complaints, worker performance, and room status. Promotional codes shall also be created, edited, activated, deactivated, or deleted; active codes shall be published in real time to the Hotel Website. Reports may be exported as PDF or CSV.
Pre-conditions	• The manager shall be authenticated on the dashboard. • Relevant operational data shall exist in the system.
Post-conditions	• Analytics and reports shall have been displayed across selected time periods. • Optional: A promotional code shall have been created, updated, or status-changed. • Active promotional codes shall be reflected on the Hotel Website in real time.
Main Flow	1. The Manager Dashboard shall be accessed; the system shall display a summary of today's bookings, occupancy rate, and pending alerts. 2. A section shall be selected: Booking & Revenue Analytics, Food Order Analytics, Service Request Analytics, Complaints Register, Worker Performance, or Room Status Board. 3. A time period (daily, weekly, or monthly) shall be selected; the system shall retrieve and display charts, heatmaps, and KPIs for the chosen section. 4. Low-booking alerts shall be displayed by the system with a recommendation to create a promotion when identified. 5. To manage promotions, the Promotions section shall be accessed; the manager shall create, edit, activate, deactivate, or delete promotional codes with discount details, validity period, applicable room types, and usage limits. 6. The updated promotion status shall be published by the system to the Hotel Website in real time. 7. Any analytics report (bookings, revenue, or staff performance) may be exported by the manager as PDF or CSV.

Table 13: Use Case Description MD-01

10.2.2 UC-MD-02 - Manage Staff Accounts

Use Case ID	UC-MD-02
Use Case Name	Manage Staff Accounts
Actor(s)	Manager
Description	Staff accounts shall be created and managed by the manager through the dashboard, covering Receptionist, Worker, and Kitchen Staff roles. Account registration, role assignment, credential generation, password reset, profile editing, and deactivation shall be supported. Credentials shall be auto-generated and delivered to staff via email.
Pre-conditions	• The manager shall be authenticated on the dashboard.
Post-conditions	• A staff account shall have been created, updated, or deactivated. • Login credentials shall have been generated and delivered to the staff member via email.
Main Flow	1. The Staff Management section shall be accessed by the manager.

	2. An operation shall be selected: Create New Account, Edit Profile, Reset Password, Change Role, or Deactivate Account. 3. For account creation, staff details including name, role (Receptionist / Worker / Kitchen Staff / Kitchen Manager), age, phone, email, and NIC shall be entered by the manager. 4. The entered details shall be validated by the system. 5. The staff profile shall be created; a username and temporary password shall be auto-generated by the system. 6. The generated credentials shall be securely sent to the staff member via email (SendGrid). 7. For password reset, the manager shall select the account; the system shall generate new credentials and dispatch them via email. 8. For profile edits or role changes, the manager shall modify relevant fields and save; the system shall update the record immediately.

Table 14: Use Case Description MD-02

10.3 Subsystem 03 - Front Desk System

10.3.1 UC-FD-01 - Perform Guest Check-in

Use Case ID	UC-FD-01
Use Case Name	Perform Guest Check-in
Actor(s)	Front Desk Staff
Description	A guest check-in shall be performed by the front desk staff for both walk-in and pre-booked guests. Guest details and identification shall be verified, a room shall be assigned, payment processed (walk-in) or confirmed (pre-booked), FOSS access activated, and the room status updated to Occupied. All actions shall be recorded in the audit log.
Pre-conditions	• The front desk staff shall be authenticated. • For walk-in: at least one available room shall exist. • For pre-booked: a valid reservation shall exist in the system.
Post-conditions	• The guest shall have been checked in and a room assigned. • The room status shall be updated to Occupied. • FOSS access shall have been activated for the guest. • The check-in action shall have been recorded in the audit log.
Main Flow	1. The arrivals panel shall be accessed; for walk-in guests, a new booking shall be created by entering guest details, room type, dates, and processing payment (cash or card). 2. For pre-booked guests, the reservation shall be retrieved by searching by name, Booking Reference ID, NIC, email, or phone. 3. Guest identity shall be verified by uploading a scanned copy or marking physical document verification as complete. 4. A specific room number shall be assigned if not pre-assigned; available rooms shall be displayed by the system. 5. The check-in record shall be confirmed; the room status shall be updated to Occupied by the system. 6. A FOSS session token linked to the room shall be activated by the system for the duration of the guest's stay. 7. A registration card and payment receipt may be printed by the receptionist.

Table 15: Use Case Description FD-01

10.3.2 UC-FD-02 - Guest Checkout, Manage Room Status, and Handle Escalated

Use Case ID	UC-FD-02
Use Case Name	Process Guest Checkout, Manage Room Status, and Handle Escalated Tasks
Actor(s)	Front Desk Staff
Description	Guest checkout shall be processed by the front desk staff, generating an itemised folio of all charges and settling final payment. The room status shall be updated and a cleaning task auto-created in WKMS. The front desk staff shall also monitor room status in real time, mark rooms Under Maintenance, create service requests on behalf of guests, and manually assign escalated tasks to available workers.
Pre-conditions	• The front desk staff shall be authenticated. • For checkout: the guest shall be checked in with outstanding charges available. • For escalation handling: tasks shall have remained unclaimed beyond 15 minutes. • For guest services: a guest shall have an active checked-in stay.
Post-conditions	• For checkout: the final bill shall have been settled; guest status updated to Checked Out; FOSS deactivated; room set to Requires Cleaning; a cleaning task created in WKMS with High priority. • For room management: room status updates shall be reflected in real time. • For task assignment: the escalated task shall be assigned to the selected worker and removed from the shared queue. • For guest services: the guest's folio shall be displayed (view path) or a service request shall be created in WKMS with the correct room number (request path).
Main Flow	1. For checkout, the guest record shall be retrieved; the system shall generate a full itemised folio including room charges, food orders, and additional services. 2. The final bill shall be reviewed and presented to the guest; payment shall be confirmed and processed. 3. The guest status shall be updated to Checked Out; the room status set to Requires Cleaning; FOSS access deactivated; a cleaning task automatically created in WKMS with High priority by the system. 4. For room status management, the room status board shall be accessed; a room shall be marked as Under Maintenance to block it from future bookings until cleared. 5. A service request may be created on behalf of a guest and submitted directly to the WKMS task queue. 6. The escalation panel shall display unclaimed tasks exceeding 15 minutes, highlighted in red; a notification badge and alert shall be shown by the system. 7. An available worker shall be selected by the receptionist; the Assign action shall be initiated; the task shall be removed from the shared queue and updated in WKMS in real time. 8. To view a guest's folio: staff searches for the guest by name or room number; system retrieves the active stay and displays the running folio showing all accumulated charges to date.

	9. To create a service request on behalf of a guest: staff selects the request category and optionally adds a description; system submits the request to the WKMS task queue with the guest's room number automatically attached and confirms creation.

Table 16: Use Case Description FD-02

10.4 Subsystem 04 - Food Ordering and Service Request App (FOSS)

10.4.1 UC-FOSS-01 - Access FOSS and Place Food Order

Use Case ID	UC-FOSS-01
Use Case Name	Access FOSS and Place Food Order
Actor(s)	Guest
Description	The FOSS application shall be accessed by the guest by scanning the room QR code. The guest session shall be authenticated using a room-linked token. The guest shall browse meal-category menus showing only available items within the active time window, add items to a cart with dietary/allergy notes, and submit a food order, which shall be forwarded to KMS. The order may be cancelled within 10 minutes if not yet in preparation.
Pre-conditions	• The guest shall have been checked in and a valid FOSS session shall be active. • A QR code linked to the room shall be available. • The selected meal category shall be within its active ordering time window.
Post-conditions	• The FOSS session shall have been authenticated. • The food order shall have been placed and forwarded to KMS. • An estimated delivery time shall have been displayed to the guest. • Optional: the order shall have been cancelled if within the 10-minute window.
Main Flow	1. The QR code in the guest room shall be scanned; the FOSS web application shall open on the guest's device. 2. The guest session shall be authenticated by the system using the room-linked token; session validity shall be verified. 3. The FOSS home interface shall load; meal category menus (Breakfast, Lunch, Dinner, Tea-Time) shall be displayed showing only available items within the active ordering window. 4. A menu item shall be selected; details including name, description, price, allergen tags, dietary labels, calorie count, and portion limit shall be displayed. 5. Items shall be added to the cart; quantities shall be specified subject to per-item portion limits. 6. Optional allergy information and dietary preferences shall be entered (up to 300 characters). 7. The order summary shall be reviewed and the Place Order action initiated; the system shall validate availability, quantity limits, and time window. 8. A unique order number and estimated delivery time shall be generated; the order shall be forwarded to KMS by the system.

	9. To cancel, the Cancel Order action shall be initiated within 10 minutes; the system shall verify eligibility and update the status to Cancelled; KMS shall be notified.

Table 17: Use Case Description FOSS-01

10.4.2 UC-FOSS-02 - Submit Service Request and Complaint

Use Case ID	UC-FOSS-02
Use Case Name	Submit Service Request and Complaint
Actor(s)	Guest
Description	A service request shall be submitted by the guest through FOSS and its status tracked in real time until completion. A complaint linked to a service request or order may also be submitted and tracked through resolution.
Pre-conditions	• The guest shall be authenticated in FOSS with an active session.
Post-conditions	• The service request shall have been created and sent to the WKMS task queue. • Real-time status tracking shall be available to the guest. • Optional: a complaint shall have been recorded and linked to the related request or order.
Main Flow	1. The Service Request section shall be accessed by the guest. 2. A request type (e.g., housekeeping, extra towels, maintenance) shall be selected; an optional description shall be entered. 3. The request shall be submitted; the system shall record it and add it to the WKMS shared task queue. 4. Real-time status updates shall be displayed by the system: Pending → In Progress → Completed. 5. To submit a complaint, the Complaints section shall be accessed; a related service request or order shall be selected. 6. Complaint details shall be entered and submitted; the system shall record the complaint and link it to the referenced request or order. 7. The complaint status shall be displayed by the system: Submitted → Under Review → Resolved.

Table 18: Use Case Description FOSS-02

10.5 Subsystem 05 - Kitchen Management System

10.5.1 UC-KMS-01 — Manage Menu Items, Stock Status, Ordering Time Limits

Use Case ID	UC-KMS-01
Use Case Name	Manage Menu Items, Stock Status, and Ordering Time Limits
Actor(s)	Kitchen Staff
Description	Menu items shall be created, updated, or removed by kitchen staff, and stock availability status shall be managed. Ordering time limits (start and cut-off times) shall be configured per meal category. All changes shall be reflected in the FOSS menu in real time.
Pre-conditions	• The kitchen staff shall be authenticated with the appropriate role.
Post-conditions	• Menu items shall have been created, updated, or removed. • Stock status shall have been set (Available, Low Stock, or Unavailable). • Ordering time limits shall have been saved and enforced in FOSS.

	• All changes shall be reflected in the FOSS menu in real time.
Main Flow	1. The Menu Management section shall be accessed by the kitchen staff. 2. A meal category shall be selected; a menu operation (add, edit, or remove) shall be chosen. 3. Menu item details including name, description, price, allergen tags, dietary labels, calorie count, preparation time, portion limit, and availability shall be entered or updated. 4. The entered details shall be validated and saved by the system; the menu shall be updated in FOSS in real time. 5. The stock status shall be set for each item as Available, Low Stock, or Unavailable; a low-stock alert shall be sent to the Manager Dashboard by the system. 6. To configure ordering time limits, the Ordering Time Settings section shall be accessed; start time and cut-off time shall be set per meal category. 7. The time settings shall be validated and saved; the system shall enforce the limits in FOSS, displaying only items within the active window.

Table 19: Use Case Description KMS-01

10.5.2 UC-KMS-02 - Receive, Process, and Update Food Orders

Use Case ID	UC-KMS-02
Use Case Name	Receive, Process, and Update Food Orders
Actor(s)	Kitchen Staff
Description	Food orders shall be received from FOSS in real time, displayed on the kitchen dashboard with audio and visual alerts, and a Kitchen Order Ticket (KOT) automatically printed. Kitchen staff shall process orders through the FIFO queue, acknowledge allergy notes, and update order status from New to In Preparation to Ready. Upon readiness, WKMS shall be notified to assign a delivery worker.
Pre-conditions	• The Kitchen Management System shall be operational. • A food order shall have been placed by a guest via FOSS. • The kitchen thermal printer shall be connected and reachable.
Post-conditions	• The order shall have been displayed on the kitchen dashboard and a KOT printed. • The order status shall have been updated through the preparation cycle. • WKMS shall have been notified to assign a delivery worker when the order is marked Ready. • For cancellations within 10 minutes: the order shall be removed from the queue and a CANCELLED KOT printed.
Main Flow	1. A food order shall be received by the system from FOSS in real time; an audio chime and visual flash notification shall be triggered. 2. The order shall be displayed on the kitchen dashboard in the New Orders queue; a KOT shall be automatically printed to the kitchen thermal printer within 3 seconds. 3. Order details including items, quantities, allergy notes, and dietary instructions shall be reviewed by the kitchen staff. 4. The order status shall be updated to In Preparation by the kitchen staff; orders in this state cannot be cancelled.

	5. Upon completion, the order status shall be updated to Ready by the kitchen staff. 6. A notification shall be sent by the system to WKMS to assign a delivery worker. 7. If a cancellation request is received from FOSS within 10 minutes and the order has not entered preparation, the order shall be marked Cancelled, removed from the queue, and a CANCELLED KOT printed. 8. The order shall be marked as Delivered by the worker via WKMS; the final status shall be reflected in both KMS and FOSS.

Table 20: Use Case Description KMS-02

10.6 Subsystem 06 - Worker Management System (WKMS)

10.6.1 UC-WKMS-01 — View Shared Task Queue and Self-Assign Task

Use Case ID	UC-WKMS-01
Use Case Name	View Shared Task Queue and Self-Assign Task
Actor(s)	Worker
Description	A shared queue of service tasks shall be displayed to workers in real time. Tasks shall be voluntarily claimed by workers subject to a maximum of three active tasks simultaneously. Once claimed, the task shall be assigned to the worker and removed from the shared queue.
Pre-conditions	• The worker shall be authenticated in WKMS. • Service tasks shall exist in the shared queue.
Post-conditions	• The selected task shall have been assigned to the worker. • The task shall have been removed from the shared queue. • The task shall be visible in the worker's My Tasks list.
Main Flow	1. The shared task queue shall be accessed by the worker; available tasks shall be displayed in real time. 2. Task details including type, room number, priority, and submission time shall be reviewed by the worker. 3. A task shall be selected; the Claim action shall be initiated by the worker. 4. The system shall verify that the worker has fewer than three active tasks; if the limit is reached, the claim shall be rejected with a notification. 5. The task shall be assigned to the worker by the system; it shall be removed from the shared queue and added to the worker's My Tasks list. 6. Tasks unclaimed for more than 15 minutes shall be automatically marked as Escalated by the system and an alert sent to the Front Desk dashboard.

Table 21: Use Case Description WKMS-01

10.6.2 UC-WKMS-02 - Update Task Status and View Shift Performance

Use Case ID	UC-WKMS-02
Use Case Name	Update Task Status and View Shift Performance
Actor(s)	Worker
Description	The status of an assigned task shall be updated by the worker as it progresses from In Progress to Completed. For food delivery tasks,

	the order shall be marked as Delivered after completion. Task completion data shall be recorded for manager analytics. The worker may also view shift performance metrics on the personal dashboard.
Pre-conditions	• The worker shall be authenticated in WKMS. • At least one task shall be assigned to the worker.
Post-conditions	• The task status shall have been updated to Completed. • For delivery tasks: the food order shall have been marked as Delivered in KMS and FOSS. • Task completion data shall have been recorded for manager analytics. • Shift performance metrics shall have been displayed.
Main Flow	1. The My Tasks section shall be accessed by the worker; assigned tasks shall be displayed. 2. A task shall be selected; the status shall be updated to In Progress by the worker. 3. Upon finishing the task, the status shall be marked as Completed by the worker. 4. For food delivery tasks, the order shall be marked as Delivered; the status shall be synchronized with KMS and FOSS by the system. 5. Task completion time and details shall be recorded by the system for manager performance analytics. 6. To view performance, the Shift Dashboard section shall be accessed; the system shall display completed tasks, average completion time, and currently active tasks for the shift.

Table 22: Use Case Description WKMS-02

CHAPTER 11: ACTIVITY DIAGRAMS

11.1 Online Room Booking

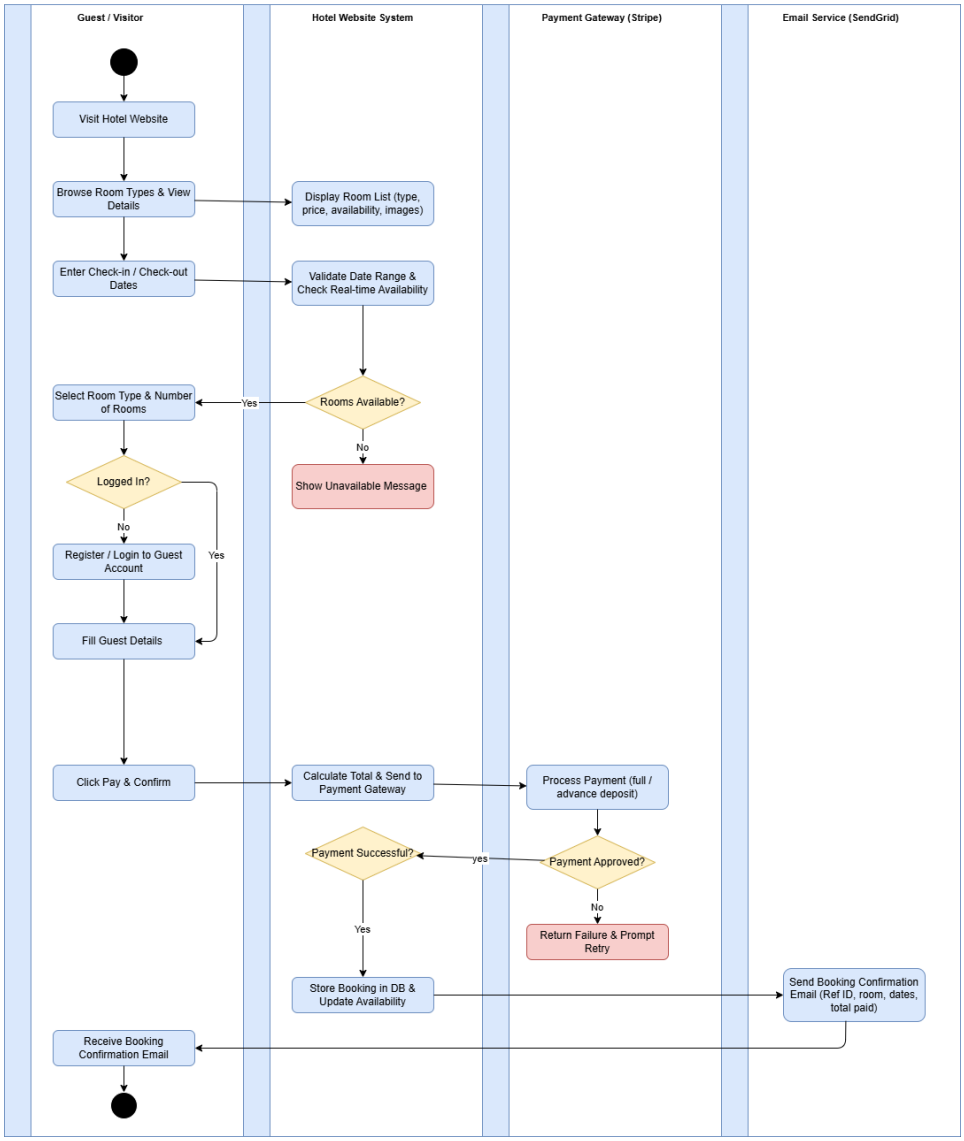


Figure 7: Activity Diagram of Online Room Booking

11.2 Guest Check-in

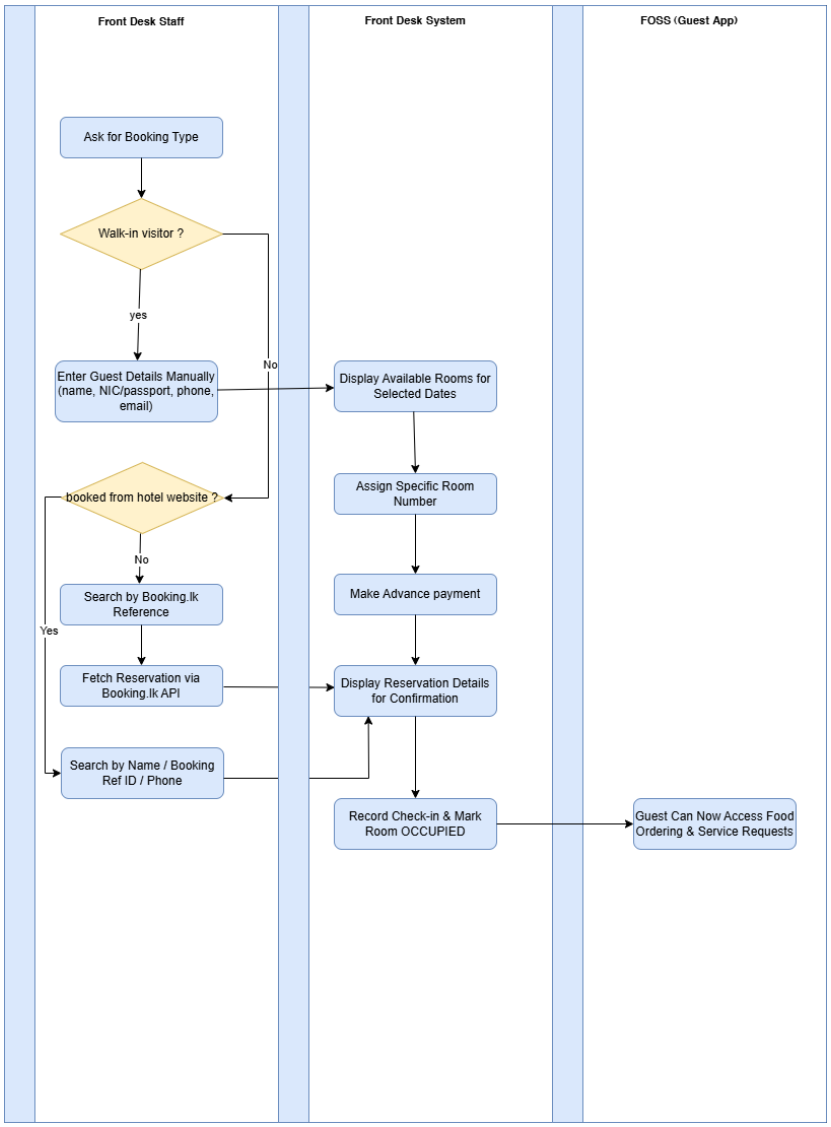


Figure 8: Activity Diagram of Guest Check-in

11.3 Guest Check-out

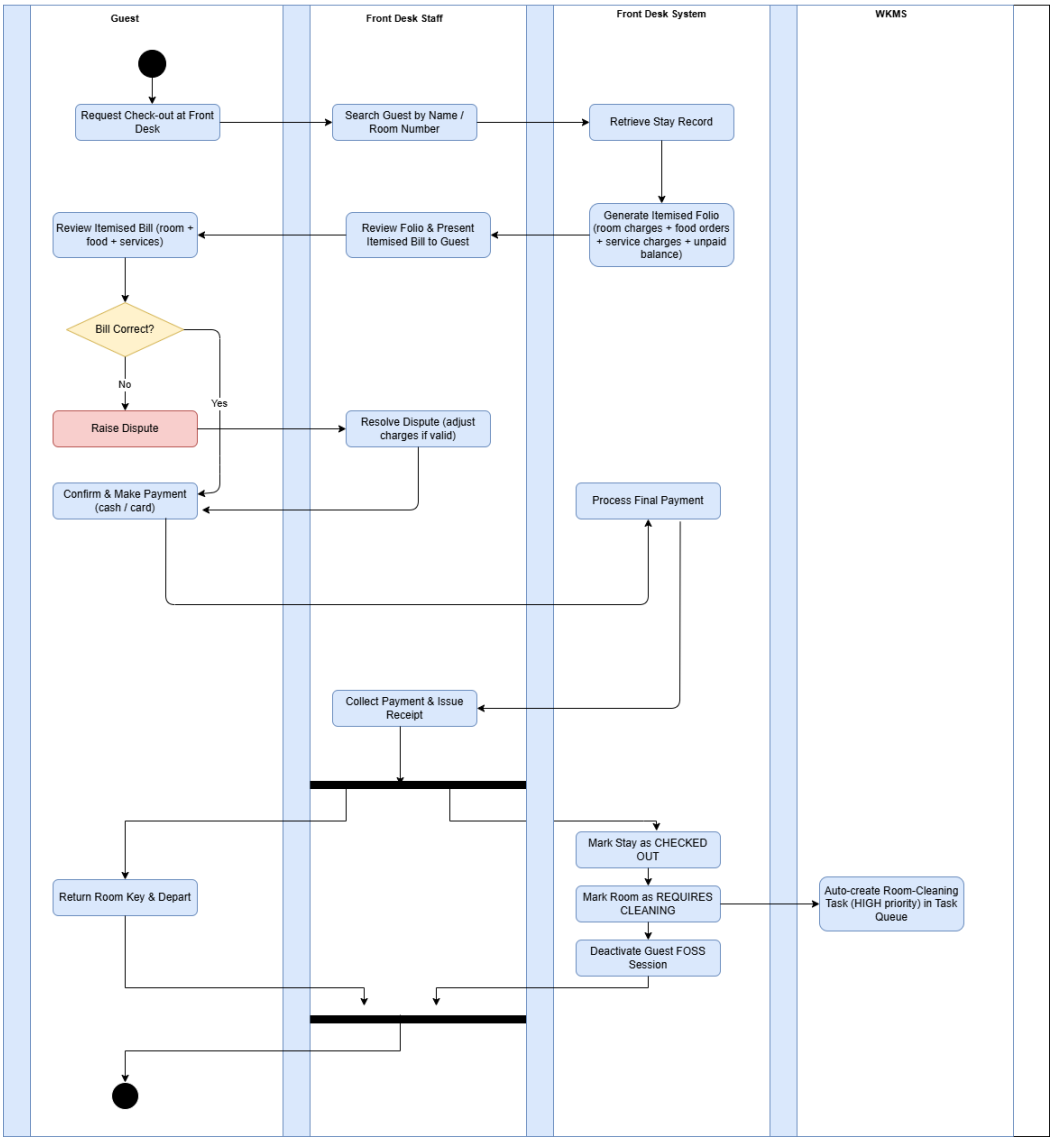


Figure 9: Activity Diagram of Guest Check-out

11.4 Worker Performance Complaint Management

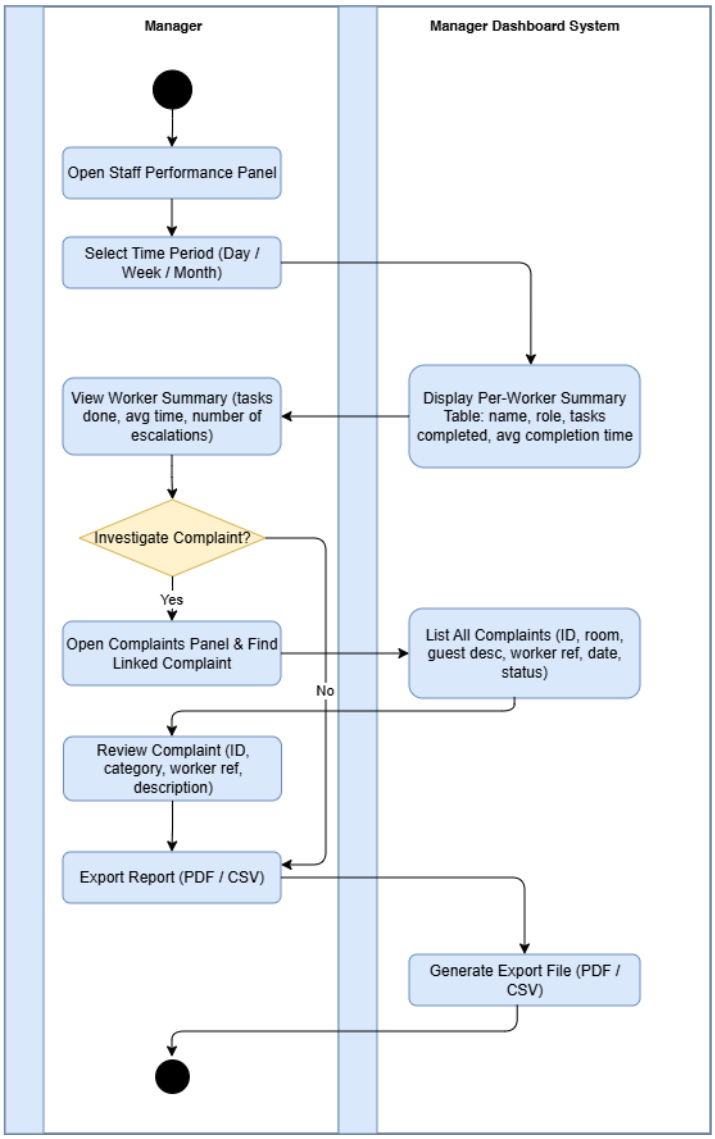


Figure 10: Activity Diagram of Worker Performance Complaint Management

11.5 Register Worker

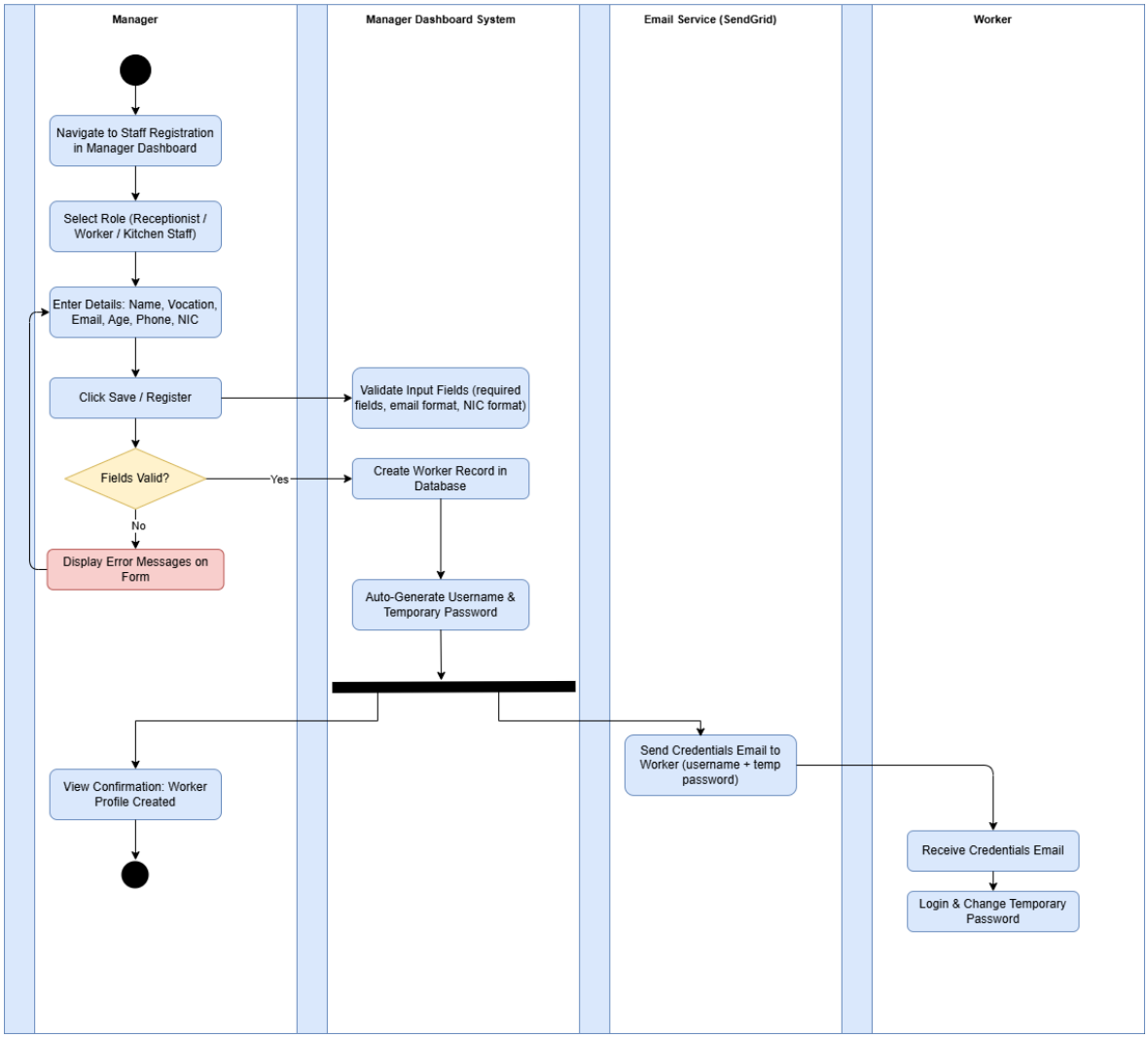


Figure 11: Activity Diagram of Register Worker

11.6 Manager Analytics

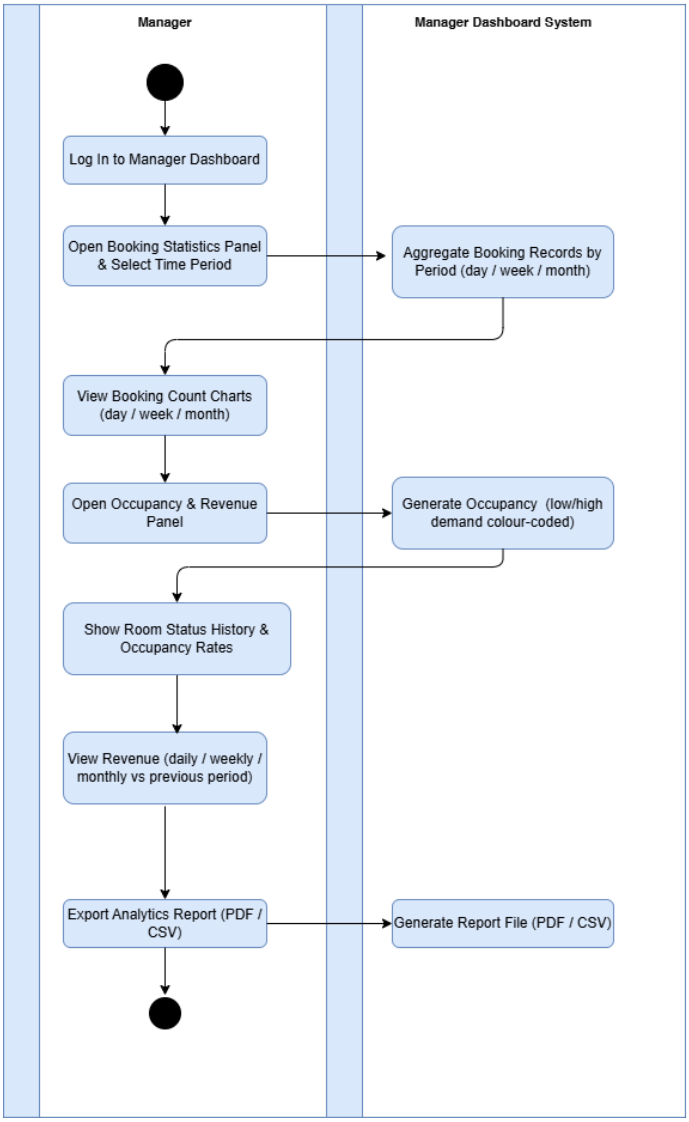


Figure 12: Activity Diagram of Manager Analytics

11.7 Manage Promotions

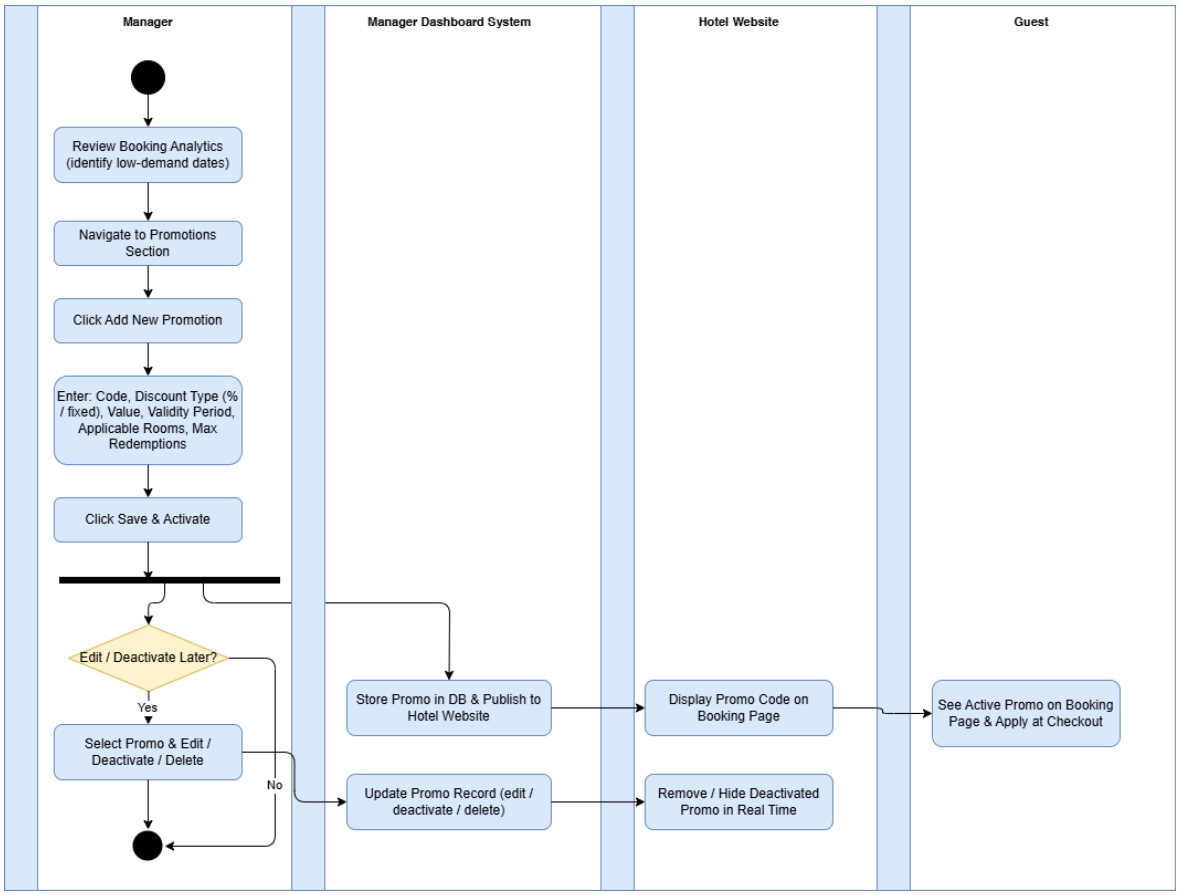


Figure 13: Activity Diagram of Manage Promotions

11.8 Service Request

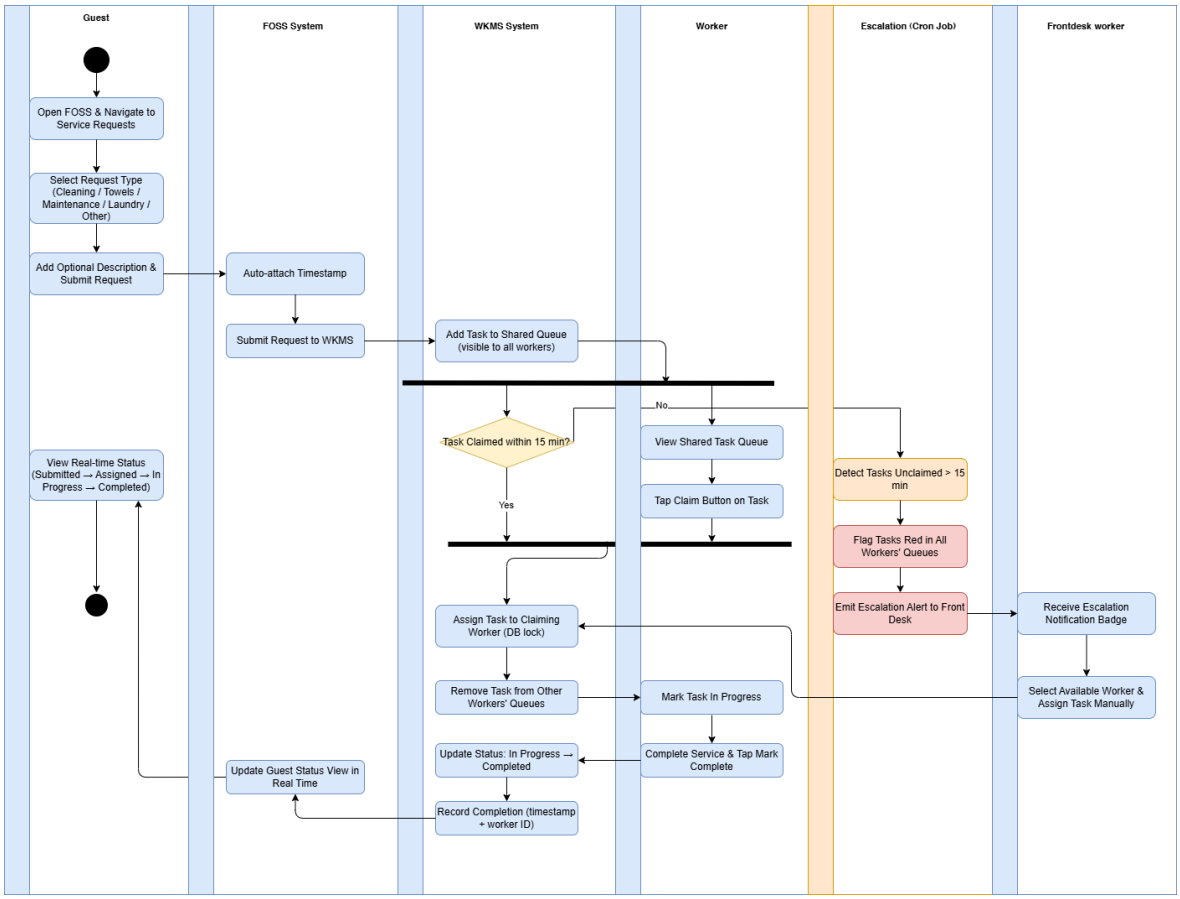


Figure 14: Activity Diagram of Service Request

11.9 Food Order

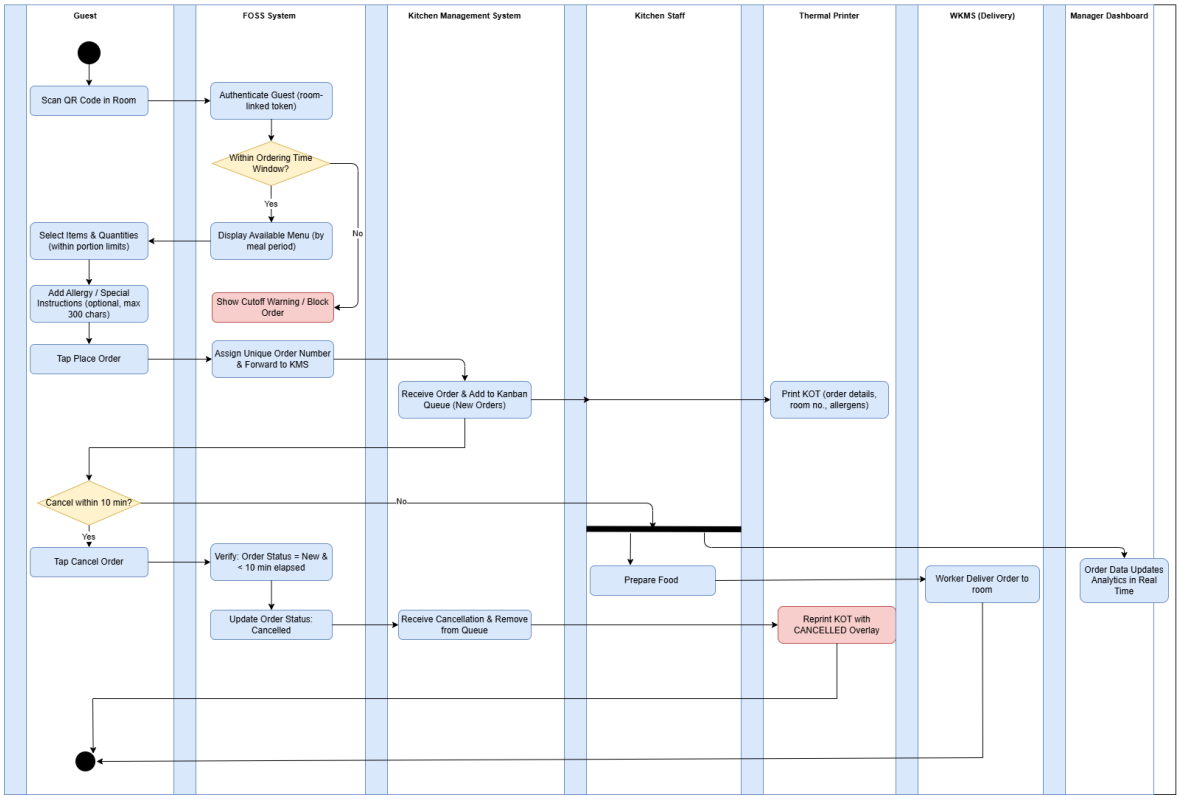


Figure 15: Activity Diagram of Food Order

11.10 Worker Task Claim

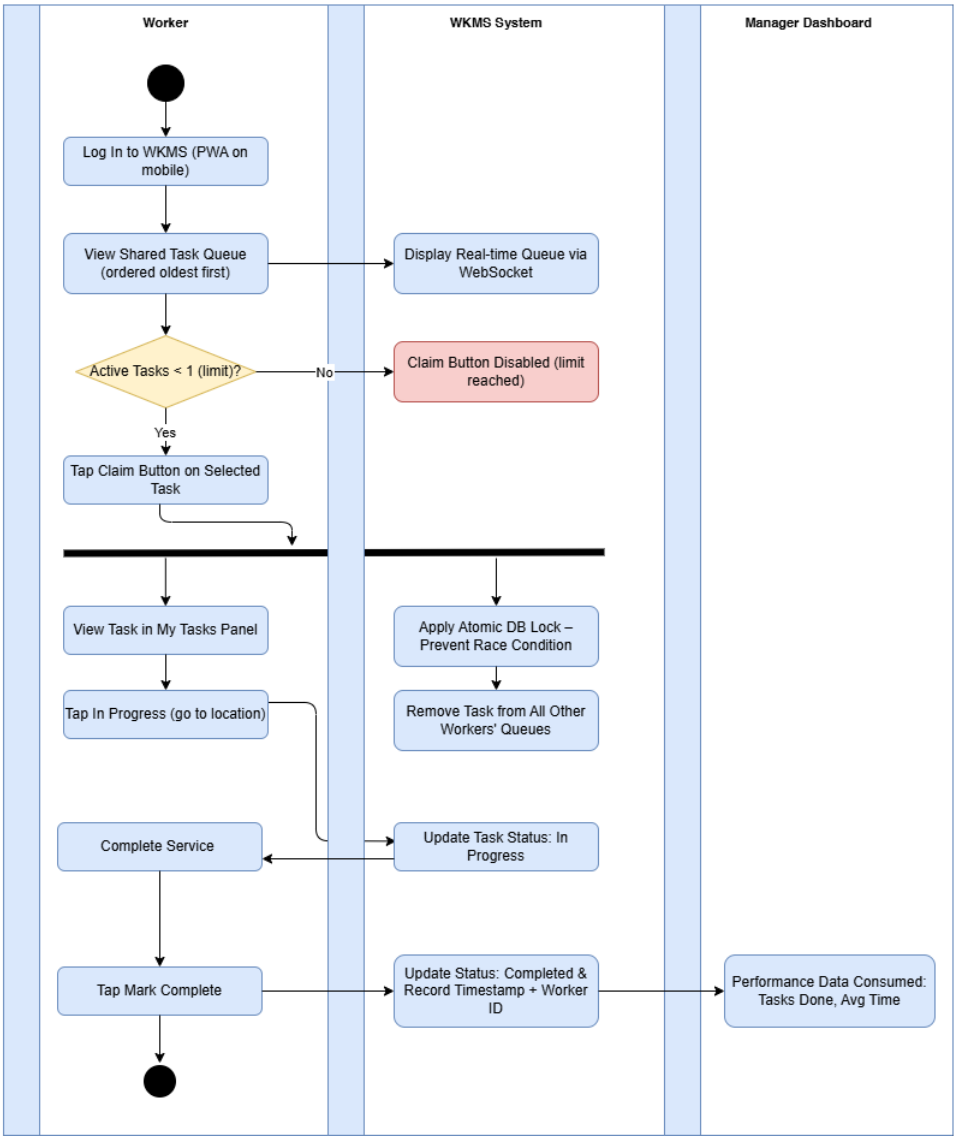


Figure 16: Activity Diagram of Worker Task Claim

11.11 Kitchen Order processing

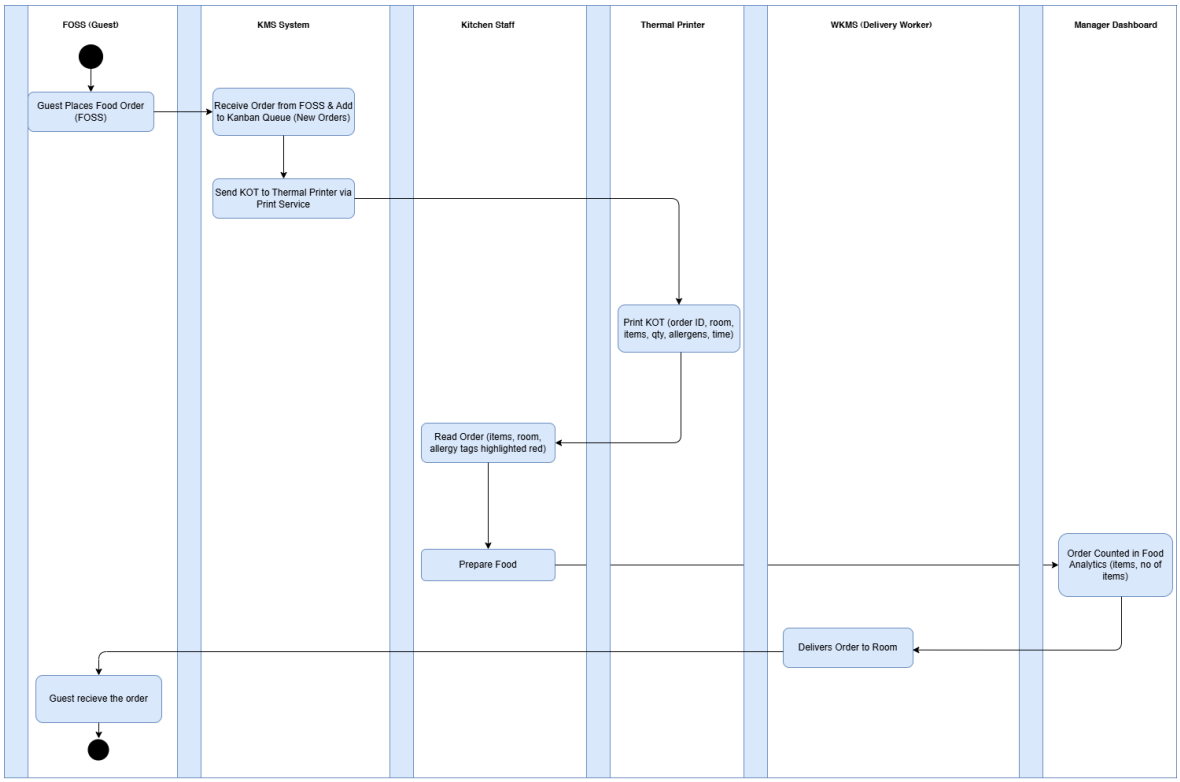


Figure 17: Activity Diagram of Kitchen Order Processing

CHAPTER 12: EXTERNAL INTERFACE REQUIREMENTS

All external interface requirements are defined in this chapter for Forever Hotel, covering interactions with users, hardware devices, third-party software services, and communication channels. These requirements establish all boundary conditions of the system in a precise and verifiable manner.

12.1 User Interface Requirements

Forever Hotel encompasses six subsystems, each serving a distinct user role. The following requirements apply across all subsystems unless otherwise specified.

12.1.1 General user interface standards

- A consistent visual design language (unified colour palette, typography - Times New Roman or equivalent system-safe serif, and component library) shall be maintained across all interfaces.
- All staff-facing interfaces (MAD, FDS, KMS, WKMS) shall be implemented as Next.js Single Page Applications (SPAs) with fast in-page transitions.
- All text, labels, error messages, and notifications shall be written in plain English; technical jargon shall be avoided in guest-facing interfaces.
- Inline validation with descriptive error messages shall be provided for all form inputs, adjacent to the relevant field, prior to submission.
- Minimum touch target size of 44 × 44 px shall be maintained per WCAG 2.1 [11].
- Loading indicators (spinners or skeleton screens) shall be shown for all async data-fetch operations exceeding 300 ms.
- Full keyboard navigation and visible focus states shall be supported across all interfaces.

12.1.2 Hotel website (HW) - public-facing interface

- The Hotel Website shall be rendered correctly on viewport widths ranging from 320 pixels (mobile) through 768 pixels (tablet) to 1280 pixels and above (desktop).
- The booking flow shall be presented as a multi-step wizard, with a visible progress indicator displaying the current step: Room Selection, Guest Details, Payment, and Confirmation.
- The room availability calendar shall employ colour coding to differentiate available dates (green), fully booked dates (red), and selected date ranges (blue highlight).

- Up to three room types shall be presented in a side-by-side comparison table with consistent attribute rows to facilitate direct feature evaluation by prospective guests.
- A cookie consent banner shall be displayed upon the user's first visit, in compliance with applicable data protection regulations.

Descriptive alternative text attributes shall be provided for all images to support screen readers and to meet WCAG 2.1 Level AA accessibility standards [11].

12.1.3 Manager dashboard (MAD)

- The Manager Dashboard shall be structured using a sidebar navigation layout, with clearly labelled sections for Analytics, Room Management, Promotions, Staff Accounts, and Notifications.
- Accessible colour choices shall be applied to all charts and data visualisations such that information is not conveyed by colour alone. Data table alternatives shall be provided for screen reader compatibility.
- The occupancy heatmap shall utilise a diverging colour scale to indicate demand levels, accompanied by a visible legend.
- Unresolved notification alerts shall be displayed persistently at the top of the dashboard, with a count badge applied to the corresponding navigation icon.
- Report export controls (PDF and CSV) shall be accessible directly from each analytics view without requiring navigation away from the current page.

12.1.4 Front desk system (FDS)

- The FDS interface shall be optimised for use on a 1920 x 1080 desktop monitor and shall employ a dual-panel layout: a left panel for arrivals and departures, and a right panel for room status.
- The room status board shall present a colour-coded grid: Green (Clean/Vacant), Blue (Occupied), Orange (Requires Cleaning), and Red (Under Maintenance).
- A red notification badge shall be displayed on the escalation panel when a new escalation event is received, accompanied by an audible chime when the browser tab is active.
- Guest folio charges shall be presented in a hierarchical itemised format grouped by category (room charges, food and beverage, and services), with subtotals provided for each category.

12.1.5 Guest app / FOSS — in-room portal

- The Guest App shall be delivered as a Progressive Web Application (PWA) accessible via QR code scan, such that no installation is required on the guest's mobile device.
- The interface shall be optimised for single-hand mobile use, with primary actions (Add to Cart, Submit Request) positioned within the lower third of the screen.
- Food menu items shall be presented as visual cards displaying a high-resolution image, item name, price, and allergen badge icons.
- The order status tracker shall employ a horizontal step indicator with animated progression through the states: Submitted, In Kitchen, Ready, and Delivered.
- Both light mode and dark mode display shall be supported, based on the guest device's system preference.

12.1.6 Kitchen management system (KMS)

- The KMS order queue shall be presented using a Kanban board layout, optimised for display on a 1080p kitchen display monitor or large tablet.
- Allergen annotations on order cards shall be rendered within a red highlighted badge to ensure immediate visibility to kitchen staff.
- New order notifications shall trigger both a visual flash overlay on the Kanban board and an audible chime, regardless of the kitchen staff member's current screen position.
- Status update controls (Move to Preparation, Mark Ready) shall be rendered at a minimum size of 120 x 44 pixels to facilitate use in a kitchen environment where staff may be wearing gloves.

12.1.7 Worker management system (WKMS)

- The WKMS shall be implemented as a Progressive Web Application accessible on any modern mobile browser over hotel Wi-Fi, without requiring installation from an application store.
- The task queue shall be rendered as a vertically scrolling card list, optimised for portrait orientation on mobile devices.
- Escalated tasks shall be highlighted with a red background and a pulsing border animation to draw immediate attention from workers.
- Haptic feedback shall be provided on the Claim button (on supported devices) upon successful task claiming.

12.2 Hardware Interface Requirements

Forever Hotel interfaces with the following on-premise and guest hardware devices. All specifications are defined in Table 38.

ID	Device	Subsystem(s)	Interface Method	Requirement
HW-IF-01	Network Thermal Printer (ESC/POS)	KMS, Printing Service	TCP/IP over LAN (port 9100); ESC/POS command protocol [12]	Kitchen Order Tickets transmitted to kitchen thermal printer via Printing Service. Printer reachable via static IP; acknowledgement confirmed within 3 seconds of order placement.
HW-IF-02	Front Desk Receipt Printer	FDS	USB or network (LPD/RAW); standard receipt format	Guest registration cards and payment receipts printed at check-in and check-out.
HW-IF-03	Guest Mobile Device (iOS / Android)	FOSS, WKMS	Wi-Fi (IEEE 802.11 b/g/n/ac) [13]; HTTPS; PWA via browser	Accessible on iOS 14+ or Android 9+ with a modern browser. No native application installation required.
HW-IF-04	QR Code Display (In-Room)	FOSS	Optical scan via guest device camera	Unique QR code with room-specific session token displayed in each room. Generated by FDS at check-in; distributed physically or digitally..
HW-IF-05	Hotel Server / Cloud Host	All subsystems	Docker containers on Linux x86-64 server	All microservices deployed as Docker containers on Linux (Ubuntu 22.04 LTS+); minimum 8 GB RAM and 4 CPU cores; continuous internet for public-facing subsystems.
HW-IF-06	Kitchen Display / Staff Workstation	KMS, MAD, FDS	Standard desktop with 1080p display; HTTPS browser	Staff-facing subsystems optimised for 1920 × 1080; accessible via Chrome 100+, Edge 100+, or Firefox 100+.

Table 23: Hardware Interface Requirements

12.3 Software Interface Requirements

Forever Hotel is required to integrate with the following external software services and internal platform components. All integrations are mediated through the centralised API Gateway unless otherwise specified.

12.3.1 External third-party service integrations

The external service integrations required by the system are detailed in Table 39.

ID	Service	Provider	Protocol / API	Purpose and Requirement
SI-01	Stripe Payment Gateway	Stripe, Inc.	HTTPS REST API; Stripe.js SDK; Stripe Webhooks [14]	Processes online room booking payments. Card data tokenised client-side via Stripe.js; raw card data never stored on Forever Hotel servers. Webhook events (payment_intent.succeeded / payment_intent.payment_failed) used to confirm or reject bookings. PCI-DSS compliant [15].
SI-02	SendGrid Email API	Twilio SendGrid	HTTPS REST API v3; JSON payload; API Key authentication [16]	Delivers transactional emails: booking confirmations, cancellation receipts, and staff credential notifications using pre-defined HTML templates. SendGrid message ID logged per email dispatched.
SI-03	Booking.lk Reservation API	Booking.lk	HTTPS REST API; JSON; API Key authentication	FDS imports guest reservations from Booking.lk via webhook or polling, mapped to the Forever Hotel booking schema. Imported reservations labelled with their source channel.
SI-04	Google Maps Embed API	Google LLC	JavaScript Maps Embed API; URL-based iframe embed [17]	Hotel location embedded on the Hotel Website About Us page using a domain-restricted API key. No user data transmitted to Google Maps.

Table 24: External Third-Party Service Integrations

12.3.2 Internal platform software interfaces

The internal platform components with which each subsystem is required to interact are specified in Table 40.

ID	Component	Protocol	Consuming Services	Requirement
SI-05	API Gateway	HTTPS REST; JWT validation; rate limiting	All client-facing subsystems	All client requests routed through the API Gateway. JWT authentication enforced on all protected routes; per-IP rate limiting applied; requests routed to downstream microservices by path prefix.
SI-06	RabbitMQ Message Broker	AMQP 0-9-1 [18]; JSON payloads; durable queues	KMS, WKMS, FDS, MAD, Analytics Service	Asynchronous inter-service events (food order ready, task escalation, stock status change, checkout completion) published and consumed via topic exchanges. All queues declared durable; dead-letter queues implemented by consumer services.
SI-07	PostgreSQL Database	PostgreSQL wire protocol; TCP port 5432; PgBouncer connection pooling [19]	All microservices	Dedicated PostgreSQL schema per microservice (database-per-service pattern). No cross-service SQL joins. Schema changes managed via version-controlled migration scripts.
SI-08	Printing Service (Microservice)	Internal REST API; ESC/POS over TCP [12]	KMS	Internal REST endpoint accepts a KOT payload and dispatches ESC/POS print commands to the kitchen thermal printer. Success or failure returned to KMS within 3 seconds.
SI-09	WebSocket Server	WebSocket Secure (WSS) over TLS [20]	FDS, MAD, WKMS, KMS, FOSS	Real-time updates (room status, task queue, order queue, escalation alerts) delivered via persistent WebSocket connections. Minimum 300 simultaneous connections supported.

Table 25: Internal Platform Software Interfaces

12.4 Communications Interface Requirements

The system shall use the following communication technologies:

12.4.1 Network protocol requirements

The network protocols to be employed by the Forever Hotel system are defined in Table 41.

ID	Protocol	Standard	Usage Context	Requirement
CI-01	HTTPS	TLS 1.2 / TLS 1.3 (RFC 8446) [21]	All client–server and external API communications	All communication encrypted via HTTPS; TLS 1.0/1.1 disabled. SSL certificates provisioned via Let's Encrypt and auto-renewed.
CI-02	WebSocket Secure (WSS)	RFC 6455 [20], over TLS	Real-time dashboard and queue updates	Order queues, task queues, and room status boards use WSS. Server-side reconnection supported; client-side exponential backoff with max 5 retry attempts.
CI-03	REST over HTTPS	HTTP/1.1 (RFC 7230) [22]; JSON (RFC 8259) [23]	All microservice APIs and third-party integrations	All microservice and external APIs expose RESTful endpoints. Request/response payloads encoded as UTF-8 JSON. All responses include Content-Type: application/json.
CI-04	AMQP 0-9-1	AMQP 0-9-1 (RabbitMQ) [18]	Asynchronous inter-service messaging	All asynchronous event-driven communication uses AMQP via RabbitMQ. Payloads are UTF-8 JSON. All exchanges and queues declared durable.
CI-05	TCP/IP (Raw Socket)	RFC 793 / RFC 791 [24]	Kitchen thermal printer communication	Printing Service communicates with kitchen printer over raw TCP (default port 9100) using ESC/POS binary protocol [12]. Printer configured with a static LAN IP.
CI-06	SendGrid HTTP API	SMTP (RFC 5321) [25]; SendGrid REST API v3 [16]	Transactional email delivery	Transactional emails dispatched via SendGrid HTTP API v3 with API key authentication. DKIM and SPF DNS records configured for the hotel's sending domain.

Table 26: Network Protocol Requirements

12.4.2 Data formats and standards

- All API request and response bodies shall be encoded in JSON (RFC 8259 [23]) using UTF-8 character encoding.
- Dates and timestamps shall be formatted in accordance with ISO 8601 [26] (e.g., 2025-07-15T14:30:00Z) across all APIs and data stores. UTC shall be used as the canonical timezone.
- Currency values shall be represented as integers in the smallest currency unit (e.g., LKR cents) to avoid floating-point precision issues, with the currency code (LKR) included in the payload.
- JWT tokens shall employ the HS256 or RS256 signing algorithm and shall include the following standard claims: iss, sub, exp, iat, and a custom role claim [27].
- RabbitMQ event messages shall conform to a defined envelope schema containing: eventType (string), sourceService (string), timestamp (ISO 8601), correlationId (UUID v4), and payload (object).

12.4.3 Authentication and security standards

- JWT bearer tokens issued by the central Auth Service shall be validated on all authenticated endpoints. Token expiry shall be set to eight hours for staff sessions and 24 hours (or until checkout) for guest sessions [27].
- Stripe.js client-side tokenisation shall be utilised for all payment processing; card data shall not traverse Forever Hotel application servers, ensuring PCI-DSS SAQ A compliance [15].
- Guest passwords on the Hotel Website shall be stored as bcrypt hashes with a minimum cost factor of 12 [28].
- Staff credentials shall be transmitted only via encrypted email as one-time-use temporary passwords, which shall be required to be changed upon first login.

CHAPTER 13: DATA REQUIREMENTS

This chapter specifies the data entities managed by the Forever Hotel system, including their key attributes, retention requirements, integrity constraints, and privacy considerations. All data requirements have been derived from the functional requirements specified in Chapters 4 and 5 of the full SRS.

13.1 Data Entities and Attributes

13.1.1 Booking

The Booking entity represents a room reservation made via the Hotel Website, the Front Desk System, or imported from a third-party channel. The attributes of the Booking entity are defined in Table 6.

Attribute	Type	Constraints	Notes
bookingId	UUID	Primary Key, Not Null, Unique	Auto-generated booking reference identifier
guestId	UUID	Foreign Key - Guest, Not Null	May be null for anonymous walk-in bookings prior to account creation
roomTypeId	UUID	Foreign Key - RoomType, Not Null	
roomNumber	VARCHAR(10)	Foreign Key - Room, Nullable	Assigned at check-in; null prior to check-in
checkInDate	DATE	Not Null	Must be earlier than checkOutDate
checkOutDate	DATE	Not Null	Must be later than checkInDate
status	ENUM	Not Null	Permitted values: PENDING, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED
totalAmount	INTEGER	Not Null, > 0	Stored in LKR cents to avoid floating-point precision errors
paymentReference	VARCHAR(255)	Nullable	Stripe PaymentIntent identifier

source	ENUM	Not Null, Default: WEBSITE	Permitted values: WEBSITE, WALK_IN, BOOKING_LK
createdAt	TIMESTAMPTZ	Not Null, Default: NOW()	UTC timestamp; auto-populated on record creation

Table 27: Booking Entity Attributes

13.1.2 Guest

The Guest entity stores the registered account details of hotel guests. The attributes of the Guest entity are defined in Table 43.

Attribute	Type	Constraints	Notes
guestId	UUID	Primary Key, Not Null, Unique	Auto-generated identifier
fullName	VARCHAR(255)	Not Null	
email	VARCHAR(320)	Not Null, Unique	RFC 5321 [15] compliant; used as the login identifier
passwordHash	VARCHAR(255)	Not Null	bcrypt hash with a minimum cost factor of 12 [18]
nicOrPassport	VARCHAR(50)	Nullable	Collected and verified at check-in by the receptionist
phone	VARCHAR(20)	Nullable	
createdAt	TIMESTAMPTZ	Not Null, Default: NOW()	UTC timestamp

Table 28: Guest Entity Attributes

13.1.3 Food order

The Food Order entity captures each order placed by a guest via the Guest App during their stay. The attributes of the Food Order entity are defined in Table 44.

Attribute	Type	Constraints	Notes
orderId	UUID	Primary Key, Not Null, Unique	
bookingId	UUID	Foreign Key → Booking, Not Null	Links the order to the guest's active stay
roomNumber	VARCHAR(10)	Not Null	Denormalised for kitchen display performance

mealPeriod	ENUM	Not Null	Permitted values: BREAKFAST, LUNCH, DINNER, TEA_TIME
items	JSONB	Not Null	Array of objects: { menuItemId, name, quantity, unitPrice }
allergyNote	VARCHAR(300)	Nullable	Guest-provided allergy information; maximum 300 characters
status	ENUM	Not Null	Permitted values: PLACED, IN_PREPARATION, READY, DELIVERED, CANCELLED
totalAmount	INTEGER	Not Null, >= 0	In LKR cents; appended to the guest folio at checkout
placedAt	TIMESTAMPTZ	Not Null	Used for FIFO queue sorting and 10-minute cancellation window enforcement
estimatedDeliveryTime	TIMESTAMPTZ	Nullable	Calculated by the KMS at the time of order placement

Table 29: Food Order Entity Attributes

13.1.4 Task (service request / housekeeping / delivery)

The Task entity records all service requests, housekeeping assignments, and food delivery tasks managed through the Worker Management System. The attributes of the Task entity are defined in Table 45.

Attribute	**Type**	**Constraints**	**Notes**
taskId	UUID	Primary Key, Not Null, Unique	
roomNumber	VARCHAR(10)	Not Null	
category	ENUM	Not Null	Permitted values: ROOM_CLEANING, EXTRA_TOWELS, WATER_BOTTLES, MAINTENANCE, LAUNDRY, FOOD_DELIVERY, OTHER
description	TEXT	Nullable	Guest- or receptionist-provided task details

priority	ENUM	Not Null, Default: NORMAL	Permitted values: HIGH (post-checkout cleaning), NORMAL
status	ENUM	Not Null	Permitted values: UNASSIGNED, ASSIGNED, IN_PROGRESS, COMPLETED, ESCALATED
assignedWorkerId	UUID	Foreign Key → StaffUser, Nullable	Null when unassigned; populated upon task claiming or manual assignment
submittedAt	TIMESTAMPTZ	Not Null	Used for FIFO ordering and 15-minute escalation timer enforcement
completedAt	TIMESTAMPTZ	Nullable	Populated upon task completion
source	ENUM	Not Null	Permitted values: GUEST_APP, FRONT_DESK, CHECKOUT_TRIGGER, KMS

Table 30: Task Entity Attributes

13.1.5 Room

Attribute	**Type**	**Constraints**	**Notes**
roomNumber	VARCHAR(10)	Primary Key, Not Null, Unique	Physical room identifier (e.g. 101, 202)
roomTypeId	UUID	Foreign Key - RoomType, Not Null	Links room to its category
floor	INTEGER	Not Null	Floor number
status	ENUM	Not Null	Permitted values: VACANT, OCCUPIED, REQUIRES_CLEANING, UNDER_MAINTENANCE
lastClearedAt	TIMESTAMPTZ	Nullable	Timestamp of last status change to VACANT
notes	TEXT	Nullable	Maintenance or housekeeping notes

Table 31: Room Entity Attributes

13.1.6 Menu Item

Attribute	Type	Constraints	Notes
menuItemId	UUID	Primary Key, Not Null, Unique	Auto-generated identifier
categoryId	UUID	Foreign Key - MealCategory, Not Null	Links item to meal category
name	VARCHAR(255)	Not Null	Display name shown in FOSS
description	TEXT	Nullable	Full item description
price	INTEGER	Not Null, > 0	In LKR cents
preparationTime	INTEGER	Not Null	In minutes; used for delivery time estimation
allergenTags	TEXT[]	Nullable	Array of allergen identifiers
dietaryLabels	TEXT[]	Nullable	e.g. vegan, gluten-free
portionLimit	INTEGER	Not Null, Default: 5	Maximum quantity per guest per order
stockStatus	ENUM	Not Null	Permitted values: AVAILABLE, LOW_STOCK, UNAVAILABLE, TEMPORARILY_DISABLED
imageUrl	VARCHAR(500)	Nullable	URL to high-resolution item image
createdAt	TIMESTAMPTZ	Not Null, Default: NOW()	UTC timestamp

Table 32: Menu Item Entity Attributes

13.1.7 Service Request

Attribute	Type	Constraints	Notes
requestId	UUID	Primary Key, Not Null, Unique	Auto-generated identifier
bookingId	UUID	Foreign Key - Booking, Not Null	Links request to the guest's active stay
roomNumber	VARCHAR(10)	Not Null	Denormalised for WKMS display
category	ENUM	Not Null	Permitted values: ROOM_CLEANING, EXTRA_TOWELS, WATER_BOTTLES, MAINTENANCE, LAUNDRY_PICKUP, OTHER
description	TEXT	Nullable	Guest-provided optional description

status	ENUM	Not Null	Permitted values: SUBMITTED, ASSIGNED, IN_PROGRESS, COMPLETED
submittedAt	TIMESTAMPTZ	Not Null	UTC timestamp; used for FIFO ordering
completedAt	TIMESTAMPTZ	Nullable	Populated upon task completion
source	ENUM	Not Null	Permitted values: GUEST_APP, FRONT_DESK

Table 33: Service Request Entity Attributes

13.1.8 Worker

Attribute	Type	Constraints	Notes
workerId	UUID	Primary Key, Not Null, Unique	Auto-generated identifier
fullName	VARCHAR(255)	Not Null	
vocation	VARCHAR(100)	Not Null	e.g. Housekeeping, Maintenance
email	VARCHAR(320)	Not Null, Unique	Used for credential delivery
phone	VARCHAR(20)	Nullable	
age	INTEGER	Nullable	
nic	VARCHAR(50)	Nullable	Collected at account creation by manager
username	VARCHAR(100)	Not Null, Unique	Auto-generated by system
passwordHash	VARCHAR(255)	Not Null	bcrypt hash with minimum cost factor of 12
isActive	BOOLEAN	Not Null, Default: TRUE	Set to FALSE when account is deactivated
createdAt	TIMESTAMPTZ	Not Null, Default: NOW()	UTC timestamp

Table 34: Worker Entity Attributes

13.1.9 Complaint

Attribute	Type	Constraints	Notes
complaintId	UUID	Primary Key, Not Null, Unique	Auto-generated identifier; displayed to guest as reference number
bookingId	UUID	Foreign Key - Booking, Not Null	Links complaint to the guest's stay
roomNumber	VARCHAR(10)	Not Null	Denormalised for manager dashboard display

category	VARCHAR(100)	Not Null	Guest-selected complaint category
description	TEXT	Not Null	Guest-entered complaint description
severity	ENUM	Not Null	Permitted values: LOW, MEDIUM, HIGH
referencedOrderId	UUID	Nullable	Optional reference to a Food Order
referencedRequestId	UUID	Nullable	Optional reference to a Service Request
referencedWorkerId	UUID	Foreign Key - Worker, Nullable	Optional reference to a specific worker
status	ENUM	Not Null	Permitted values: SUBMITTED, UNDER_REVIEW, RESOLVED
resolutionNotes	TEXT	Nullable	Internal manager notes; not visible to guest
submittedAt	TIMESTAMPTZ	Not Null, Default: NOW ()	UTC timestamp
resolvedAt	TIMESTAMPTZ	Nullable	Populated when status changes to RESOLVED

Table 35: Complaint Entity Attributes

13.2 Data Retention Requirements

The data retention periods specified in Table 46 shall be enforced by the system. All retention periods have been determined in accordance with financial audit requirements, data protection regulations, and operational needs.

Data Category	**Retention Period**	**Rationale**	**Action After Retention Period**
Booking Records	7 years	Financial audit and tax compliance requirements	Guest personal data shall be anonymised; aggregated financial data shall be retained for compliance.
Guest Account Data	3 years after last stay	Guest relationship management; applicable legal obligations	A soft delete with anonymisation shall be applied upon written request.
Food Order Records	2 years	Revenue reporting and menu analytics	Records shall be archived to cold storage.

Task and Service Records	1 year	Staff performance analytics and complaint investigation	Records shall be archived to cold storage.
Audit Logs (FDS actions)	3 years	Tamper-evident operational record of front desk activity	Read-only archival shall be applied; deletion is not permitted.
System Logs (structured)	12 months	Debugging and incident investigation (NFR-17)	Automated purge shall be applied after 12 months.
Payment Transaction Data	7 years	PCI-DSS compliance [15]; financial audit requirements	Raw card data shall not be stored. Only Stripe reference identifiers shall be retained.

Table 36: Data Retention Requirements

13.3 Data Integrity and Privacy Requirements

- All foreign key relationships shall be enforced at the database level using PostgreSQL foreign key constraints with appropriate ON DELETE and ON UPDATE policies [19].
- created_at and updated_at timestamp columns shall be included in all tables and shall be populated automatically via database triggers or ORM lifecycle hooks.
- Guest identification documents (NIC/passport scans uploaded during check-in) shall be stored in encrypted object storage. Access shall be restricted to authenticated front desk staff only.
- Personally identifiable information (PII) - including guest name, email, phone number, and NIC - shall not be included in RabbitMQ event payloads or system logs. References shall employ opaque UUID identifiers only.
- All database connections shall be encrypted in transit using TLS. Database credentials shall be managed via environment variables or a secrets manager and shall not be hardcoded in source code.
- A right-to-erasure workflow shall be supported: upon a guest's written request, personal data shall be anonymised (name replaced with a REDACTED token, email replaced with a unique hashed identifier) while non-personal booking financial records are retained for compliance purposes.

CHAPTER 14: ALTERNATIVE SOLUTIONS CONSIDERED

Prior to the finalisation of the Forever Hotel architecture, several alternative technical approaches were evaluated. This chapter documents the alternatives that were considered, the trade-offs identified, and the rationale underlying each design decision that was adopted.

14.1 Architecture: Monolithic vs Microservices

The architectural style of the system was evaluated by comparing a traditional monolithic application against a microservices architecture, as summarised in Table 47.

Dimension	Monolithic Architecture	Microservices Architecture (Adopted)	Decision Rationale
Development Complexity	Lower initial complexity; a single unified codebase is maintained.	Higher initial complexity; independent service repositories are required.	Justified by the long-term maintainability and independent deployability benefits.
Fault Isolation	A single point of failure exists; a crash in one module affects the entire system.	Service failures are isolated; other services continue operation (NFR-06).	Critical for 24/7 hotel operations — a kitchen failure must not affect front desk operations.
Scalability	The entire application must be scaled to address any individual bottleneck.	Individual high-load services may be scaled independently (e.g., FOSS during meal periods).	Enables targeted scaling during peak operational periods.
Independent Deployment	A full redeployment is required for any change to the system.	Individual subsystems may be deployed independently without system-wide downtime.	Allows hotfixes and feature releases to be applied to a single service.

Table 37: Architecture: Monolithic vs Microservices

The microservices architecture was adopted. The six subsystems (HW, MAD, FDS, FOSS, KMS, WKMS) are deployed as independently containerised services, with cross-cutting concerns such as authentication and rate limiting handled by the API Gateway.

14.2 Real-Time Communication: Polling vs WebSockets vs Server-Sent Events

Three approaches to real-time communication were evaluated for use in the system's order queue, task queue, and escalation alert features, as summarised in Table 48.

Approach	Advantages	Disadvantages	Verdict
Short Polling (HTTP every N seconds)	Simple to implement; widely supported across clients.	High server load is generated; stale data exists between polls; unsuitable for time-critical updates.	Rejected - Unsuitable for the kitchen order queue and task escalation requirements.
Long Polling	Lower request frequency compared to short polling.	Stateless connections remain complex to manage server-side; gaps arise when connections drop.	Rejected - Complexity is introduced without the efficiency benefits of WebSockets.
Server-Sent Events (SSE)	Simpler implementation than WebSockets; native browser support; well-suited to server-to-client streams.	Unidirectional only; a separate HTTP request is required for client-to-server messages.	Considered for the Manager Dashboard analytics view; not selected so that a single communication standard could be maintained.
WebSockets (Adopted)	Full-duplex communication; persistent connection; low latency; efficient for high-frequency bidirectional updates.	More complex to implement; connection management and reconnection logic are required.	Adopted - Ideal for order queue, task queue, and escalation alert updates.

Table 38: Real-Time Communication Approaches Compared

14.3 Task Assignment Model: Automatic Assignment vs Self-Assignment Queue

- Option A - Automatic Assignment Algorithm: Tasks are automatically dispatched to the least-loaded available worker using a round-robin or workload-balancing algorithm.
  - Advantages: Even workload distribution is ensured; no delay is introduced from workers failing to notice available tasks.
  - Disadvantages: Worker proximity, specialisation, and current physical location cannot be accounted for by the algorithm; tasks may be assigned to workers unable to accept them.

- Option B - Self-Assignment Queue with Escalation Fallback (Adopted): Workers voluntarily claim tasks from a shared queue. Tasks that remain unclaimed after 15 minutes are escalated to the front desk for manual assignment.
    - Advantages: Workers may self-select tasks appropriate to their location and current capacity; the approach is aligned with the hotel's existing operational culture; the escalation mechanism ensures no tasks are abandoned.
    - Disadvantages: Uneven distribution may arise if claim rates differ between workers; escalation monitoring is required.

The self-assignment model was selected as it provides greater operational flexibility and is better aligned with the hotel's staff autonomy preferences. The 15-minute escalation timer combined with front desk manual assignment provides a reliable safety net.

14.4 Guest In-Room Access: Native Mobile App vs Progressive Web App

Two approaches to delivering the in-room guest portal were evaluated, as summarised in Table 49.

Dimension	Native Mobile App (iOS + Android)	Progressive Web App / PWA (Adopted)	Decision Rationale
Guest Installation Friction	Installation is required via the App Store or Google Play; guests are unlikely to install an application for a short stay.	No installation is required; the app is accessed via QR code scan within seconds.	A critical differentiator: guests are not expected to install an application for a stay of one to two nights.
Development Effort	Separate iOS and Android codebases are required.	A single Next.js codebase operates across all platforms.	Development time and maintenance cost are significantly reduced.
Offline Capability	Full native offline support is available.	Partial offline access via Service Worker caching is supported; sufficient for viewing order history.	Service Worker caching meets the limited offline requirement of the Guest App.
Platform Consistency	Full native UI components provide an optimal feel.	Browser-rendered interfaces are slightly less native in feel.	An acceptable trade-off given the simplicity of the use case (menu browsing, ordering, service requests).

Table 39: Guest Access: Native App vs Progressive Web App

14.5 Payment Gateway: Stripe vs Local Payment Processors

- PayHere (Sri Lanka local gateway): Local payment methods including credit/debit cards and mobile wallets are supported. However, developer tooling is more limited, webhook infrastructure is less robust, and PCI-DSS compliance documentation is less comprehensive than that provided by Stripe.
- Stripe (Adopted): Industry-leading developer documentation is provided; Stripe.js client-side tokenisation ensures PCI-DSS SAQ A compliance without server-side card handling; a comprehensive webhook system enables reliable payment event handling [14]; and proven reliability at scale has been demonstrated.

Stripe was selected on the basis of its superior developer experience, security architecture, and alignment with the project's PCI-DSS compliance requirement [15].

CHAPTER 15: FEASIBILITY STUDY

This chapter is presented with a structured feasibility assessment of the Forever Hotel project across four dimensions: Technical, Economic, Operational, and Schedule. The purpose is to confirm project viability and document key risks with mitigations prior to development commitment.

15.1 Technical Feasibility

Assesses whether the required technology stack, infrastructure, and integrations are sufficiently mature, available, and within the development team's capability.

15.1.1 Technology stack assessment

Component	Technology	Maturity and Availability	Risk Assessment
Backend Services	NestJS (Node.js) [29]	Highly mature; LTS support; large community; well-documented microservices support.	LOW — NestJS is production-proven for microservices architectures.
Frontend Applications	Next.js [30]	Industry standard; extensive ecosystem; strong team familiarity.	LOW — The most widely used frontend library.
Database	PostgreSQL 15 [19]	Production-grade; ACID compliant; strong JSONB support for semi-structured menu data.	LOW — An industry-standard relational database with broad deployment history.
Message Broker	RabbitMQ [18]	Mature AMQP broker; widely deployed in microservices architectures; good NestJS integration.	LOW-MEDIUM — Correct queue and exchange configuration is required.
Containerisation	Docker + Docker Compose [31]	Industry standard; well-supported on Linux servers.	LOW — Standard tooling with broad community support.
Real-Time (WebSocket)	Socket.IO (NestJS gateway)	Mature library; built-in NestJS WebSocket gateway support.	LOW — Proven technology; minor configuration adjustments required for scale.
Thermal Printing	ESC/POS over TCP [12]	ESC/POS is an established industry	MEDIUM — Physical printer testing and on-site

		standard; Node.js libraries are available.	IP configuration are required.
PWA (WKMS, FOSS)	Next.js + Service Worker API [32]	Fully supported in modern mobile browsers; standardised W3C API.	LOW — Well-understood technology; minor iOS Safari compatibility considerations apply.
Third-Party APIs	Stripe [14], SendGrid [16], Booking.lk	Stripe and SendGrid provide highly mature SDKs. Booking.lk API availability requires confirmation.	MEDIUM — Booking.lk API documentation must be verified before FDS integration development commences.

Table 40: Technology Stack Assessment

15.1.2 Technical feasibility conclusion

The Forever Hotel technology stack comprises entirely mature, well-documented, open-source or commercially available technologies. No novel or experimental technologies are required. The primary technical risks are identified as follows:

- Booking.lk API availability and documentation completeness - mitigated by early API access verification as a project prerequisite.
- ESC/POS thermal printer integration - mitigated by on-site hardware testing during the development phase using the target printer model.
- RabbitMQ configuration and dead-letter queue management - mitigated by team training and phased integration testing.

Overall Technical Feasibility Rating: HIGH.

15.2 Economic Feasibility

Economic feasibility assesses whether the system can be developed and operated within a justifiable cost structure relative to the benefits delivered to Forever City Hotel.

15.2.1 Development cost estimate

- Development Team: LKR 0 (academic project - developed by students at the University of Kelaniya).
- Cloud Hosting (VPS, 1 year): LKR 120,000 – 180,000 (estimated for a 4-core / 8 GB RAM Linux VPS on DigitalOcean or equivalent).
- SSL Certificate: LKR 0 (Let's Encrypt, free and auto-renewed).
- Stripe Processing Fees: Approximately 2.9% plus a fixed fee per transaction (variable; no fixed platform cost).

- SendGrid Transactional Email: LKR 0 – 5,000 per month (free tier at 100 emails/day is anticipated to be sufficient).
- Domain Name Registration: LKR 5,000 – 8,000 per year.
- Thermal Printer (if not existing): LKR 25,000 – 45,000 (one-time hardware cost; may be existing hotel equipment).

15.2.2 Expected benefits

- Online Booking Revenue: Hotel Website enables direct bookings, reducing reliance on third-party platforms (typically 15–25% commission). A conservative increase of 10 bookings/month at LKR 15,000 average generates LKR 150,000/month in incremental direct revenue.
- Operational Efficiency Gains: Digitisation of check-in, food ordering, housekeeping, and kitchen workflows estimated to reduce staff coordination time by 30–40%.
- Reduced No-Shows and Cancellations: Automated confirmation emails and structured cancellation policy enforcement expected to reduce ad-hoc cancellations.
- Data-Driven Decision Making: Manager Dashboard enables evidence-based promotional decisions, demand-based pricing, and staff performance management - improving occupancy and revenue optimisation.

Overall Economic Feasibility Rating: HIGH - Operating costs are low relative to the anticipated revenue and efficiency benefits for a hospitality establishment of this scale.

15.3 Operational Feasibility

Operational feasibility assesses whether the system can be effectively adopted and used by the hotel's staff and management within their existing operational context.

15.3.1 Stakeholder readiness assessment

The digital literacy, adoption risk, and mitigation strategy for each primary stakeholder group are assessed in Table 51.

Stakeholder	Digital Literacy	Adoption Risk	Mitigation Strategy
Hotel Manager (STK-01)	Moderate - comfortable with basic digital tools.	LOW	The Manager Dashboard is designed with clear analytics visualisations; an individual onboarding session is planned.

Receptionist (STK-02)	Moderate - familiar with basic booking tools.	LOW-MEDIUM	The FDS is designed to mirror familiar front desk workflows; a step-by-step check-in flow is provided; a training session is included in the rollout plan.
Kitchen Staff (STK-05)	Variable - some staff may have limited digital experience.	MEDIUM	The KMS Kanban interface is visually intuitive with large controls; physical KOT printing is retained as a familiar physical reference; a kitchen-specific training session is planned.
Workers (STK-06)	Low to moderate - mobile devices may be used primarily for personal use.	MEDIUM	The WKMS is a mobile PWA with a simple card-based interface; QR code access eliminates login friction; a short hands-on training session is planned.
Hotel Guests (STK-03)	High - modern smartphones are assumed to be carried.	LOW	FOSS is a zero-installation QR-scan web application; it is designed for intuitive use without training.

Table 41: Stakeholder Readiness Assessment

15.3.2 Operational risk factors and mitigations

- Network Dependency: All subsystems are dependent on hotel Wi-Fi. This risk is mitigated through WKMS PWA offline caching for temporary outages, and reliable connectivity is assumed.
- Printer Dependency: Kitchen operations are dependent on thermal printers. This risk is mitigated by using the digital KMS order queue as a backup, with printer issues being monitored through alerts.
- Change Management: Resistance to new systems may be encountered. This risk is mitigated through a phased rollout, temporary parallel use of manual processes, and staff training supported by management.

Overall Operational Feasibility Rating: HIGH - Existing workflows are maintained with minimal disruption.

Schedule feasibility assesses whether the Forever Hotel system can be designed, developed, tested, and delivered within the available project timeline.

15.4.1 Development timeline overview

The proposed development timeline is summarized in Table 52. A total of approximately 23 weeks is required from requirements sign-off to final submission.

Phase	Activities	Duration	Deliverables
1	Requirements elicitation and SRS authoring	3 weeks	Approved SRS document; stakeholder sign-off obtained.
2	System architecture and database design	2 weeks	Architecture diagrams; ER diagrams; API contract definitions; Docker Compose scaffold.
3	Core infrastructure setup	1 week	API Gateway, RabbitMQ, PostgreSQL, and Auth Service deployed in Docker.
4	Subsystem development (parallel streams)	10 weeks	All six subsystems (HW, MAD, FDS, FOSS, KMS, WKMS) functionally complete.
5	Integration testing	2 weeks	End-to-end test cases executed; cross-service integration verified.
6	User acceptance testing (UAT) and bug resolution	2 weeks	UAT sign-off obtained from hotel management; critical defects resolved.
7	Deployment and staff training	1 week	System operational in the hotel environment; staff training sessions delivered.
8	Project documentation and submission	2 weeks	Final SRS, design documents, test reports, and deployment guide submitted.
Total	—	~23 weeks	Fully deployed, tested, and documented Forever Hotel system.

Table 42: Development Timeline Overview

15.4.2 Schedule risk assessment

- Booking.lk API Integration (FDS): If API access or documentation is delayed, the import feature will be postponed to a post-launch release without affecting core functionality.
- On-Site Hardware Testing (Thermal Printer): Testing is dependent on access to the printer environment at Forever City Hotel. It is scheduled for Phase 4 (Week 6), with buffer time allocated.

- UAT Feedback Volume: If major issues are identified during UAT, the bug resolution phase may be extended. This risk is mitigated through early walkthrough testing with hotel staff at the end of Phase 4.
- Parallel Subsystem Development: The risk of integration delays is minimized by ensuring that all API contracts are finalized in Phase 2 before development is started.

15.4.3 Schedule feasibility conclusion

The 23-week timeline is considered achievable due to the parallel development approach and clear subsystem boundaries. The critical path (API Gateway, Auth Service, and core database schema) is completed in Phases 2 and 3 before subsystem development begins. Identified risks have mitigations and do not impact on the critical path.

Overall Schedule Feasibility Rating: HIGH - The project can be completed within the academic timeline.

15.5 Feasibility Study Summary

The overall feasibility assessment across all four dimensions is presented in Table 53.

Dimension	Rating	Key Strength	Primary Risk
Technical	HIGH	Mature, well-supported technology stack throughout.	Booking.lk API availability; ESC/POS printer on-site testing.
Economic	HIGH	Low operating costs relative to clear direct booking revenue impact.	Stripe transaction fees are variable; hosting costs increase at scale.
Operational	HIGH	Workflow-aligned design with intuitive, role-specific interfaces.	Kitchen and worker staff adoption; training programme effectiveness.
Schedule	HIGH	Parallel development across well-defined subsystem boundaries; 23-week timeline.	Booking.lk API access delay; volume of UAT feedback.
Overall Verdict	APPROVED FOR DEVELOPMENT	All four feasibility dimensions return a HIGH rating. The Forever Hotel project is recommended to proceed to development.	All identified risks are manageable and are accompanied by documented mitigation strategies.

Table 43: Feasibility Study Summary

CHAPTER 16: RISK ANALYSIS

The following Risk Register documents all identified risks with their likelihood, impact, risk level, and mitigation strategies.

16.1 Risk Register

Risk ID	Description	Likelihood	Impact	Risk Level	Mitigation Strategy
R001	Failure of the kitchen printer may prevent staff from receiving food orders.	Medium	High	High	Provide a digital order dashboard as a backup if printing fails.
R002	Network failures may prevent guests from accessing the QR-based Guest App.	Medium	Medium	Medium	Ensure reliable hotel Wi-Fi and allow manual order entry by staff.
R003	Integration issues with external APIs such as Stripe or Booking.lk may delay development.	Medium	High	High	Perform early integration testing and implement fallback procedures.
R004	Changing system requirements during development may affect project scope.	Medium	Medium	Medium	Use requirement review meetings and version-controlled documentation.
R005	Staff may require time to adapt to the new digital system.	Low	Medium	Low	Provide staff training sessions and user guides before deployment.
R006	QR code misconfiguration may lead to incorrect room access or session errors.	Low	High	Medium	Use secure room-token mapping and validation checks.
R007	Failure of real-time communication	Medium	Medium	Medium	WebSocket connections are configured with

	(WebSocket) may delay task and order updates.				client-side exponential backoff and a maximum of five reconnection attempts (CI-02). Real-time dashboards display a connectivity warning banner during disconnection. Staff are trained to refresh manually if the banner persists.

Table 44: Risk Register

16.2 Risk Categories

16.2.1 Technical risks

Technical risks involve issues related to software development, system integration, and infrastructure.

Examples include:

- Failure of hardware components such as kitchen printers, which may prevent order tickets from being printed.
- System integration challenges between subsystems (FOSS, WKMS, KMS, Front Desk, Manager Dashboard).
- Real-time communication failures between modules due to messaging or network problems.
- Performance issues under high user load, especially during peak hotel occupancy.
- Database failures or data inconsistency caused by unexpected system crashes.
- Software bugs or configuration errors that may affect system functionality.

Mitigation includes proper system testing, fallback mechanisms, monitoring tools, and scalable microservices architecture.

16.2.2 Schedule risks

Schedule risks occur when project tasks take longer than expected and delay the development timeline.

Examples include:

- Underestimation of development effort for complex features.

- Delays in integration of external services such as payment gateways or booking APIs.
- Additional time required for debugging, testing, and performance optimisation.
- Changes in project scope or late requirement modifications.
- Team members being unavailable due to academic workload or other commitments.

Mitigation includes clear project planning, milestone tracking, and regular progress reviews.

16.2.3 Resource risks

Resource risks arise when the required human resources, tools, or infrastructure are insufficient.

Examples include:

- Limited experience with selected technologies such as NestJS, Next, or Docker.
- Lack of testing devices or server environments during development
- Hardware limitations such as insufficient worker mobile devices or kitchen printers
- Limited availability of technical support or system maintenance staff.
- Budget limitations affecting software tools or hosting infrastructure.

Mitigation includes proper resource planning, training, and efficient use of available tools and infrastructure.

16.2.4 Requirements risks

Requirements risks occur when system requirements are unclear, incomplete, or frequently changed.

Examples include:

- Misunderstanding of hotel operational workflows during system design.
- Changes in business rules such as food ordering or task assignment policies.
- Incomplete documentation of functional or non-functional requirements.
- Stakeholders request new features during late development stages.
- Communication gaps between developers and stakeholders.

Mitigation includes maintaining a well-documented SRS, frequent requirement reviews, and stakeholder communication.

16.2.5 External and organizational risks

External and organizational risks originate from factors outside the development team's control.

Examples include:

- Internet connectivity failures affect system access.
- Third-party API failures such as payment gateways or booking platforms
- Changes in hotel management policies or operational procedures.
- Staff resistance or difficulty adapting to new digital systems.
- Unexpected organizational changes affecting project priorities.

Mitigation includes backup operational procedures, reliable service providers, and staff training programs.

CHAPTER 17: REFERENCES

[1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[2] M. Richards, Microservices vs. Service-Oriented Architecture. Sebastopol, CA, USA: O'Reilly Media, 2015.

[3] IEEE, IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. New York, NY, USA: IEEE, 1998.

[4] Stripe, "Stripe API Reference." [Online]. Available: https://stripe.com/docs

[5] Twilio Inc., "SendGrid Documentation." [Online]. Available: https://docs.sendgrid.com

[6] VMware, Inc., "RabbitMQ Documentation." [Online]. Available: https://www.rabbitmq.com/documentation.html

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[8] ISO/IEC, Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[9] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, Internet Engineering Task Force (IETF), Aug. 2008. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc5246

[10] Vercel, "Next.js Documentation." [Online]. Available: https://nextjs.org/docs

[11] W3C Web Accessibility Initiative, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C, W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[12] Epson America Inc., "ESC/POS Command Reference," Epson Technical Reference Guide, Epson America Inc., 2019. [Online]. Available: https://download.epson-biz.com/modules/pos/index.php?page=single_doc&tno=543

[13] IEEE, "IEEE Standard for Information Technology — Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," IEEE Std 802.11-2020, IEEE, 2020.

[14] Stripe, Inc., "Stripe API Reference," Stripe Documentation, 2024. [Online]. Available: https://stripe.com/docs/api

[15] PCI Security Standards Council, "Payment Card Industry Data Security Standard (PCI DSS) v4.0," PCI SSC, Mar. 2022. [Online]. Available: https://www.pcisecuritystandards.org

[16] Twilio SendGrid, "SendGrid API Reference v3," SendGrid Documentation, 2024. [Online]. Available: https://docs.sendgrid.com/api-reference

[17] Google LLC, "Maps Embed API Overview," Google Maps Platform Documentation, 2024. [Online]. Available: https://developers.google.com/maps/documentation/embed/overview

[18] VMware, Inc., "RabbitMQ Documentation," RabbitMQ, 2024. [Online]. Available: https://www.rabbitmq.com/documentation.html

[19] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/15/

[20] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455

[21] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF, RFC 8446, Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446

[22] R. Fielding et al., "Hypertext Transfer Protocol — HTTP/1.1," IETF, RFC 7230, Jun. 2014. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7230

[23] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," IETF, RFC 8259, Dec. 2017. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8259

[24] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc793

[25] J. Klensin, "Simple Mail Transfer Protocol," IETF, RFC 5321, Oct. 2008. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc5321

[26] International Organization for Standardization, "ISO 8601-1:2019 Date and Time — Representations for Information Interchange," ISO, 2019.

[27] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[28] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in *Proc. USENIX Annual Technical Conf.*, Monterey, CA, USA, 1999, pp. 81–92.

[29] NestJS Contributors, "NestJS Documentation," 2024. [Online]. Available: https://docs.nestjs.com

[30] Meta Platforms, Inc., "Next Documentation," 2024. [Online]. Available: https://react.dev

[31] Docker, Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com

[32] MDN Web Docs Contributors, "Progressive Web Apps (PWAs)," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

The undersigned confirms that the Software Requirement Specification (SRS) has been reviewed and approved. This approval indicates agreement with the project's scope, system requirements, and defined features described in this document.

Any modifications to these requirements after approval must be submitted as a formal Change Request and must receive approval from both the client representative and the project supervisor.

Client Representative Sign-off

Name	J. Disanayak
Title / Position	General Manager
Organization	Forever Hotels (Pvt) Ltd
Date	2026/02/13
Signature	

Project Supervisor Sign-off

Name	
Position	Project Supervisor
Date	
Signature	

Appendices

Appendix A: Diagrams

All the diagrams can be accessed using this link.

https://drive.google.com/drive/folders/1rf5zm40lK_UD2M__GMEO-tdu5WrxonlM?usp=sharing